

데이터사이언스개론 팀 프로젝트 9조 최종 보고서

9조 유정욱, 김준규, 배연진

1장. 팀 소개 및 팀원별 역할 분배

- 팀원 : 김준규, 배연진, 유정욱(팀장)
- 역할 분배

유정욱(팀장) : 데이터 분석 및 프로그램 구현, 보고서 작성
김준규 : 데이터 분석 및 프로그램 구현, 보고서 작성
배연진 : 데이터 분석 및 프로그램 구현, PPT 제작

2장. 주제 선정 배경

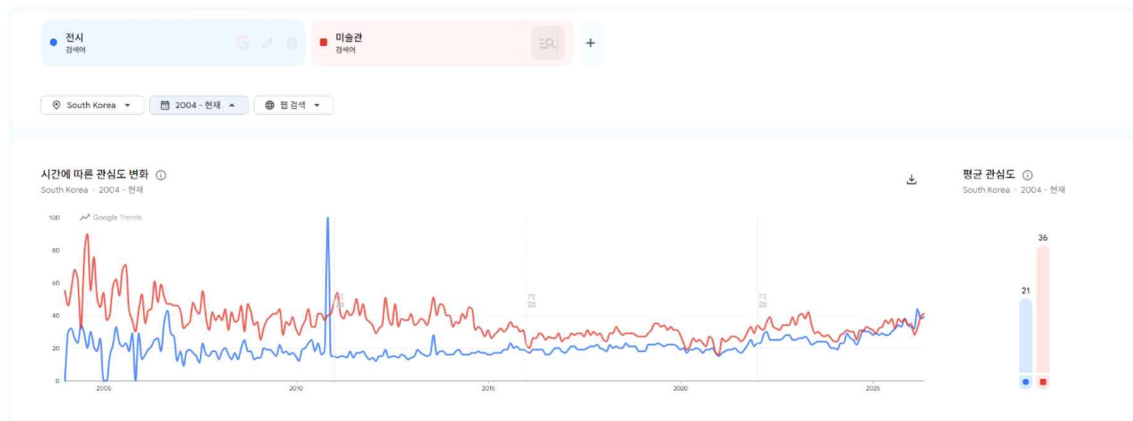
2-1. 주제

“TF-IDF 기반 비정형 텍스트 분석을 활용한 사용자 맞춤형 전시 큐레이션 웹 서비스 구축”

2-2. 주제 선정 배경

최근 문화예술에 대한 대중의 관심이 증가하면서 개최되는 전시의 수 또한 빠르게 늘어나고 있다. 구글 트렌드 분석 결과에 따르면, ‘전시회’, ‘팝업스토어’, ‘미술관’ 등 오프라인 문화 공간에 대한 검색량이 최근 몇 년간 지속적으로 우상향하는 흐름을 보인다. 그러나 전시가 과도하게 많아지면서, 관람객이 자신의 취향에 맞는 전시를 탐색하고 선택하는 과정은 오히려 더 피로하고 어려워지고 있다. 전시회는 다른 문화 콘텐츠에 비해 티켓 가격이 높고 공간적 이동이 필수적이므로 소비자가 탐색에 실패하였을 때 겪는 비용과 시간의 손실이 비교적 큰 편이다.

<구글 트렌드 분석 결과>



: ‘전시회’, ‘미술관’ 키워드 검색 추이(2004~)

영화나 드라마의 경우 넷플릭스와 같은 플랫폼을 통해 사용자의 시청 이력과 선호도를 바탕으로 한 추천 서비스가 비교적 활발하게 제공되고 있다. 반면 전시는 수요가 점차 증가하고 있음에도 불구하고, 관람객의 취향에 맞는 전시를 추천해주는 서비스는 아직 상대적으로 부족하다고 느껴졌다.

특히 ‘전시’라는 도메인은 단순한 장르적 구분을 넘어 작가의 의도, 색감, 전시의 형태 등 미묘한 ‘분위기’와 ‘키워드’가 관람객의 만족도를 결정하는 핵심 요소로 작용한다. 따라서 단순

한 조건 검색을 넘어, 전시의 비정형 설명 데이터를 분석하여 관람객의 숨겨진 취향까지 저격할 수 있는 데이터 기반의 큐레이션 시스템으로 이 문제를 해결하고자 한다.

따라서 본 프로젝트에서는 전시 설명 데이터에 포함된 키워드와 분위기를 분석하여, 과거 전시와 유사한 현재 전시를 추천하는 데이터 기반 큐레이션 프로그램을 제작하고자 하였다. 이를 통해 관람객이 직접 많은 전시 정보를 찾아보지 않아도, 자신의 관심사와 비슷한 전시를 보다 쉽게 발견할 수 있도록 돕는 것을 목표로 하였다.

2-3. 컴퓨팅 사고

분해: 전시 정보 문장들을 통째로 분석하는 대신, 전시의 주관적인 분위기 나타내는 개별 단어 단위로 쪼개어 컴퓨터가 계산할 수 있는 데이터로 변환

패턴 인식: 쪼개진 단어 데이터를 분석하여, 특정 전시마다 공통으로 많이 사용하거나 자주 함께 등장하는 단어들의 규칙을 발견

추상화: 비슷한 의미의 단어들을 컴퓨터가 쉽게 분류하고 기준을 잡을 수 있도록, 통일된 대표 키워드로 묶어 깔끔하게 정리

알고리즘 설계: 정리된 키워드 데이터를 바탕으로, "과거에 전시의 키워드"와 "현재 열리고 있는 전시의 예상 키워드"를 비교해 일치도가 높은 전시를 순서대로 추천해 주는 계산 규칙을 제작

3장. 프로젝트 목표

종료된 과거 전시의 소개 및 설명 데이터를 텍스트 마이닝하여 전시의 특성을 나타내는 핵심 키워드(top_tags)와 장르(genres)를 추출한다. 이를 바탕으로 관람객의 취향 프로파일(벡터)을 구축하고, 현재 진행 중인 전시 데이터와 수학적으로 연계하여 개인 맞춤형 전시 추천 시스템을 제안하는 것이 목표이다. 이를 통해 관람객의 탐색 피로도를 줄이고, 상대적으로 주목받지 못했던 전시로의 관람객 유입을 유도하는 실질적인 효과를 기대한다.

4장. 분석 목적 및 가치

4-1. 분석 목적

본 프로젝트의 1차적 목적은 과거 전시 경험을 토대로 현재 진행 중인 전시를 매칭해주는 추천 시스템의 뼈대를 만드는 것이다. 분석의 신뢰도와 데이터의 질을 확보하기 위해 범위를 국립현대미술관(MMCA)의 과거 전시 95개와 현재 진행 중인 전시 14개로 한정하여 데이터 분석을 진행하였다.

4-2. 분석 가치

본 프로젝트의 진정한 가치는 단순히 완성된 웹 서비스를 배포하는 것을 넘어, “주관적인 인간의 취향을 어떻게 수학적 데이터로 변환하고 가중치를 부여할 것인가”를 치열하게 고민한 분석 과정 그 자체에 있다.

전시 데이터를 단순히 병합하는 것에 그치지 않고, 전시의 뼈대가 되는 ‘장르(Genre)’보다 구체적 분위기를 나타내는 ‘핵심 태그(Top_tags)’에 더 높은 가중치(비율 3:7)를 부여하는 하

이브리드 텍스트 스코어링 방식을 직접 설계하였다. 이는 텍스트 데이터의 빈도를 의도적으로 조작하여 추천 알고리즘의 정교함을 높이는 의미 있는 데이터 전처리 과정이었다.

5장. 선행 연구 분석 및 연구 방향성

성공적인 큐레이션 시스템 구축을 위해 대표적인 콘텐츠 추천 플랫폼인 ‘왓차피디아(Watcha Pedia)’의 추천 알고리즘 및 선행 연구를 분석하고, 이를 ‘전시’ 도메인에 맞게 변형하여 본 연구의 방향성을 설정하였다.

5-1. 왓차피디아 분석 및 시사점

왓차피디아는 약 6.5억 개의 평점 데이터를 바탕으로 사용자의 탐색 시간을 줄여주고, 취향을 기록하게 만들어 데이터를 다시 생산하게 하는 선순환 구조를 띠고 있다. 선행 연구¹⁾에 따르면 이러한 큐레이션 서비스의 성공 비결은 세부적인 카테고리나 키워드에 대한 평가를 통해 사용자의 선호도를 정교하게 파악하는 데 있다.

그러나 왓차는 유저 간의 방대한 평점 데이터를 기반으로 한 ‘협업 필터링(Collaborative Filtering)’ 방식을 주로 사용한다. 이 방식은 과거 사용자의 로그 데이터가 많을수록 성능이 뛰어나지만 누적 기록이 없는 신규 전시나 유저에게는 추천 정확도가 떨어지는 ‘콜드 스타트(Cold Start)’ 문제가 필연적으로 발생한다.

5-2. 본 연구의 차별화된 방향성 (전시 도메인 맞춤형 설계)

우리는 전시라는 도메인의 특성과 데이터 확보의 현실적 한계를 고려하여, 왓차의 방식에서 나아가 다음과 같은 차별화된 연구 방향성을 설정하였다.

1. 콜드 스타트 방어를 위한 콘텐츠 기반 필터링(CBF) 채택 :

신규 전시의 경우 실관람객의 평점 데이터를 즉각적으로 확보하기 어렵다. 따라서 우리는 유저 간의 평가가 아닌, ‘전시 자체에 제공되는 소개 글에서 추출된 텍스트(장르, 태그)’를 기반으로 유사도를 측정하는 콘텐츠 기반 필터링(Content-based-filtering)을 채택하여 콜드 스타트 문제를 원천적으로 방어하였다.

2. ‘전시’의 특성을 반영한 TF-IDF 및 가중치 모델링 :

전시는 영화와 달리 관람 시간, 공간의 분위기, 작가의 철학 등 비정형적 텍스트 정보가 감상에 절대적인 영향을 미친다. 우리는 정제된 데이터의 단어 빈도서를 바탕으로 단어의 중요도를 평가하는 TF-IDF 기법을 사용하였다. 특히 왓차피디아가 ‘태그’를 중요시하는 점에 착안하여 사용자가 선택한 과거 전시의 ‘핵심 태그’ 데이터를 내부적으로 3배 증폭시키는 가중치 설계를 통해 텍스트 마이닝의 한계를 극복하고자 하였다.

3. 코사인 유사도(Cosine Similarity)를 통한 정량적 매칭 :

추출 및 가중치가 부여된 사용자의 취향 텍스트 문치와 현재 전시들의 텍스트 데이터를 다차원 공간의 벡터로 변환한 뒤, 코사인 유사도를 사용하여 그 방향성(각도)이

1) OTT서비스의 콘텐츠 추천 기능 사용자경험 개선 연구 -넷플릭스(Netflix)와 왓차(Watcha)를 중심으로-, 2019, 손보람/최종훈

가장 일치하는 전시를 도출하였다. 이는 주관적인 느낌(취향)을 철저히 정량적인 수치로 환산하여 결과를 도출해 낸 중요한 분석 과정이다.

6장. 가설 및 예상 결과

6-1. 가설

"과거에 열렸던 전시들의 정보를 분석해 키워드를 뽑아내고, 이를 현재 열리고 있는 전시 정보와 연결한다면, 관람객들은 자신이 인지하지 못했던 취향에 맞는 전시를 발견하게 될 것이다."

6-2. 예상 결과

본 프로젝트를 통해 과거 전시의 정보를 분석하여 전시의 분위기를 나타내는 핵심 키워드를 추출하고, 정제된 데이터를 바탕으로 현재 진행 중인 새로운 전시 정보와 연결하여, 사용자의 숨겨진 취향에 맞는 전시를 골라주는 맞춤형 추천 시스템의 뼈대를 완성할 것으로 예상된다.

6-3. 예상되는 한계

전시회 정보 문장을 단순한 단어 추출 방식만으로 완벽하게 파악하기는 어려우며, 특정 범위로 좁혀 데이터 분석을 진행하기에 프로토타입 수준으로 분석을 검증해야 한다는 한계점이 존재한다.

7장. 분석 데이터 및 분석 도구

7-1. 분석 데이터

- 사용한 오픈 API: 문화공공데이터광장 “한국문화정보원 외_전시정보(통합)”
- 오픈 API 주소:

<https://www.culture.go.kr/data/openapi/openapiView.do?id=598&category=l&keyword=%EB%AF%B8%EC%88%A0&searchField=all&gubun=B>

- 비정형데이터: 국립현대미술관(<https://www.mmca.go.kr/>) 전시 설명

7-2. 분석 도구

- 분석 환경 및 개발 도구: Colab, Jupyter Notebook, Cursor(free version)
- 사용한 생성형 AI: Gemini(pro version)
- 웹사이트 구현 플랫폼: 아임웹

8장. 데이터 분석 과정

8-1. 오픈 API 반정형데이터 -> 정형데이터로 가공

문화공공데이터광장에서 제공하는 오픈 API인 “한국문화정보원 외_전시정보(통합)”을 활용신청하여 사용하였으며, 제공받은 서비스를 받아 오픈 API에 접근하였다.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<response>
  <header>
    <resultCode>0000</resultCode>
    <resultMsg>OK</resultMsg>
  </header>
  <body>
    <item>
      <title>가나다락-글놀이 말놀이</title>
      <cntc_inst_nm>국립민속박물관</cntc_inst_nm>
      <collected_date>2026-05-05 00:10:03</collected_date>
      <issued_date>2026-04-30</issued_date>
      <description>전시 운영 관련 시간: 09:00~18:00(17:30 입장 마감) 단체 관람 문의: (전자우편 이용) • 단체 관람은 매주 수, 금요일 10명 이상 30명 이하인 가능합니다. 관람 일주일 전까지 문의해주시기 바랍니다. • 주시 공간이 별도로 마련되어 있지 않습니다. 단체 관람을 이용해 주시기 바랍니다. 전시 개요 전시 제목: 가나다락-글놀이 말놀이 전시 기간: 2026.5.13(수) ~ 2026.8.30(일) 전시 장소: 국립민속박물관 기획전시실 2 전시 소개 국립민속박물관은 2026년 한글날(가가날) 100주년을 맞아 국립민속박물관과 함께 '말글'·'놀이'를 주제로 특별전을 준비했습니다.<title>가나다락-글놀이 말놀이</title>는 한글을 아는 글자 '가나다라'에 놀이의 즐거움을 더해가는 특별한 여정입니다. 소리가 곧 모양이 되고 자음과 모음을 조합해 글자를 만드는 한글은 그 자체로 훌륭한 놀이 도구가 됩니다. 우리는 놀이를 통해 한글의 새로운 모습과 유연한 변화를 마주하기도 합니다. 이번 전시는 '읽고 쓰는 문자'가 아닌 우리가 가진 '가장 자유로운 놀이 도구'로서의 한글을 만나는 시간입니다. 한글과 다른 문자들의 놀이 비교부터 한글 지오 조합을 통한 글자 설계, 감각적인 소리 체험과 함께 해독까지 한글이 가진 무한한 놀이의 가능성을 펼쳐 보입니다. '한글'이라는 체계적인 순계를 바탕으로 마련한 이 놀이장에서 어린이의 말글과 자식을 마음껏 발휘해 보시기 바랍니다. 놀이를 아는 말 'OO' 놀이를 사을 여기 여기 있어라!' '무궁화꽃이 피었습니다!' 이런 시정, 놀이타를 가득 채웠던 말들을 기억하시나요? 잠 알 골목도, 너를 은둔장도 순식간에 놀이터로 바꾸는 마법, 바로 놀이를 아는 '말'이었습니다. 놀이를 쫓고, 사람을 얻고, 즐거움을 아는 '우리말'의 활기찬 힘을 느껴보세요. 큰 가리기 구름, 기워 바워 보 등 놀이의 시작과 규칙을 만드는 생생한 말을 감각적인 영상으로 소개합니다. 1부 말글 놀이 자장소 1부 <title>말글 놀이 자장소</title>에서는 한글을 읽고 재미있게 익히는 배움의 놀이와 한글로 유쾌를 즐겼던 문화가 담긴 기록을 만나 보았습니다. 나아가 배움의 시작으로부터 순서대로 디지털 환경까지 매체의 변화에 발맞추어 끊임없이 확장되어 온 말글 놀이의 세계를 소개합니다. 아울러 박물관에서 디지털 환경으로 이어지는 말글 놀이의 여정을 따라가며, 어린이의 상상력에 자극을 받은 유쾌한 언어 감각을 깨워주세요. 2부 말글 놀이 공작소 <title>말글 놀이 공작소</title>는 이곳을 한글의 구조적 원리를 놀이로 재해석하는 창의적인 실험 공간입니다. 한글을 비롯해 한자, 로마자가 가진 문자의 특성을 놀이로 경험하고 한글의 자음과 모음을 조합하여 다양한 글자를 설계해 보세요. 소리 문자 한글의 특성을 살린 유쾌한 소리 놀이와 초성·중성·종성을 모아쓰는 세계를 활용한 원로 해독까지 다채로운 재료가 기다리고 있습니다. 한글의 구조 속에서 펼쳐지는 무한한 놀이의 가능성을 만나보고, 어린이의 변혁하는 말글로 공작소를 가득 채워주세요.</description>
      <image_src>
        <local_id>666</local_id>
        <url>https://www.hangeul.go.kr/exhibition/666</url>
      </image_src>
      <view_count>95</view_count>
      <subdescription>
        <spatial_coverage>
          <event_site>국립민속박물관 기획전시실 2</event_site>
        </spatial_coverage>
        <genre>
          <duration>
            <number_pages>
              <table_of_contents>
                <author>
                  <contact_point>
                    <actor>
                      <contributor>
                        <audience>
                          <price>
                            <period>
                              <event_period>2026-05-13 ~ 2026-08-30</event_period>
                            </period>
                          </event_period>
                        </audience>
                      </contributor>
                    </actor>
                  </contact_point>
                </author>
              </table_of_contents>
            </number_pages>
          </duration>
        </genre>
      </subdescription>
    </item>
  </body>
</response>
```

오픈 API 접근 시, 화면의 데이터는 XML 형식의 반정형데이터로 나타났으며, 이를 분석에 용이하게 <item> 단위로 행을 만들고 각 태그를 열로 정리해 CSV/엑셀 형태의 정형데이터로 가공하는 과정을 거쳤다.

이때, 실제 분석에 활용할 <TITLE>, <CNTC_INST_NM>, <COLLECTED_DATE>, <DESCRIPTION>으로 한정지어 데이터를 추출하였다. 초기에는 Google Colab 환경에서 OpenAPI를 활용하여 데이터를 수집하고자 하였다. 그러나 API 서버에 대한 DNS 오류가 발생하여 정상적인 연결이 이루어지지 않았다. 이에 실행 환경을 Anaconda 기반의 Jupyter Notebook으로 변경하여 API를 호출하였으며, 이후 데이터 수집에 성공하였다.

```
import requests
import pandas as pd
import math
import time
```



```
url = "https://api.kcisa.kr/openapi/API_CCA_145/request"
service_key = "723711b7-c40f-492c-a4a6-f63e6bfc0468"
rows_per_page = 100
all_items = []
```

0단계: 서버에서 데이터를 대량으로 긁어와, 파이썬으로 분석하도 데이터베이스화하기 위해 데이터 수집 파이프라인을 구축하는 시작점에 해당하는 코드다.

try:

```
# 1. API 설정 및 총 개수 확인
params = {'serviceKey': service_key, 'numOfRows': rows_per_page, 'pageNo': 1}
headers = {'accept': 'application/json'}

response = requests.get(url, params=params, headers=headers)
data = response.json()
total_count = int(data['response']['body']['totalCount'])
total_pages = math.ceil(total_count / rows_per_page)

print(f"📄 전체 데이터 {total_count}개 수집을 시작합니다. (총 {total_pages}페이지)")

# 2. 모든 데이터 수집
for i in range(1, total_pages + 1):
    params['pageNo'] = i
    res = requests.get(url, params=params, headers=headers)
    if res.status_code == 200:
        items = res.json()['response']['body']['items']['item']
        if isinstance(items, list):
            all_items.extend(items)
        else:
            all_items.append(items)

    # 진행 상황을 10페이지마다 한 번씩 깔끔하게 출력
    if i % 10 == 0 or i == total_pages:
        print(f"[{i}/{total_pages}] 수집 중... (누적: {len(all_items)}개)")
        time.sleep(0.1)
```

1~2단계: 전체 데이터가 몇 개인지 확인한 뒤, 1페이지부터 마지막 페이지까지 100건씩 받아 [all_items]라는 하나의 큰 주머니에 모두 담는 과정을 구현한 코드다.

```
# 3. 데이터프레임 변환 및 컬럼 정제
df = pd.DataFrame(all_items)
df.columns = df.columns.str.lower()
df = df[['title', 'description', 'cntc_instt_nm', 'period']]

# 빈칸을 "null"이라는 글자로 채우기
df = df.fillna("null")

# 4. 줄바꿈 기호 제거 (엑셀 표 박살 방지용 미반의 코드)
df['title'] = df['title'].str.replace('\n', ' ', regex=False).str.replace('\r', ' ', regex=False)
df['description'] = df['description'].str.replace('\n', ' ', regex=False).str.replace('\r', ' ', regex=False).str.replace('\t', ' ', regex=False)

# 5. 엑셀 파일로 저장
output_file = "culture_data_final_clean.xlsx"
df.to_excel(output_file, index=False)

print("-" * 30)
print(f"✅ 수집 및 정제 완료! 파일명: {output_file}")
```

3~5단계: 수집된 데이터를 표로 만들어서 필요한 컬럼만 추출한 뒤, 엑셀 칸이 깨지지 않도록 노이즈를 정리하여 최종 엑셀 파일로 저장하는 코드다.

```
import pandas as pd

# 1. 아까 성공적으로 만든 전체 엑셀 파일 불러오기
# (만약 파일명이 다르다면 괄호 안의 이름을 알맞게 수정해 주세요)
file_name = "culture_data_final_clean.xlsx"
df = pd.read_excel(file_name)

# 2. ★ 핵심: 'cntc_insttt_nm' 기둥에서 '국립현대미술관'인 데이터만 쏙 골라내기
# (마지 엑셀에서 돋보기로 검색하는 것과 같은 원리입니다.)
df_filtered = df[df['cntc_insttt_nm'] == '국립현대미술관']

# 3. 골라낸 데이터가 몇 개인지 확인하기
print(f"👉 국립현대미술관 데이터: 총 {len(df_filtered)}건 찾았습니다!")
print("-" * 30)

# 4. 새로운 엑셀 파일로 저장하기
output_file = "culture_data_mmca.xlsx"
df_filtered.to_excel(output_file, index=False)

print(f"✅ 필터링 완료! 오직 국립현대미술관 데이터만 담긴 '{output_file}' 파일이 생성되었습니다.")

# 5. 잘 골라졌는지 상위 5개 미리보기
display(df_filtered.head())
```

국현미 데이터 추출: 앞서 정제 완료한 데이터 엑셀 파일을 다시 읽어봐, 기관명이 ‘국립현대미술관’으로 등록된 데이터만 찾아내어 해당 정보만 담긴 별도의 맞춤형 엑셀 파일로 분리하는 코드다.

```
import pandas as pd

# 1. 파일 불러오기
file_name = "culture_data_mmca.xlsx"
df = pd.read_excel(file_name)

# 중복 제거 전 데이터 개수 확인
print(f"👉 중복 제거 전 데이터 수: {len(df)}개")

# 2. ★ 핵심: 'title' 기둥을 기준으로 중복된 데이터 제거하기
# subset=['title'] : "제목이 같은지 검사해라!"
# keep='first' : "중복된 것들을 발견하면, 가장 먼저 나온(위에 있는) 것 1개만 살려라!"
df_unique = df.drop_duplicates(subset=['title'], keep='first')

# 중복 제거 후 데이터 개수 및 지워진 개수 확인
removed_count = len(df) - len(df_unique)
print(f"👉 중복 제거 후 데이터 수: {len(df_unique)}개")
print(f"📄 완전히 똑같은 이름을 가진 중복 데이터 {removed_count}개가 깔끔하게 삭제되었습니다!")
print("-" * 30)

# 3. 중복이 제거된 깨끗한 데이터를 새로운 엑셀 파일로 저장하기
output_file = "culture_data_unique.xlsx"
df_unique.to_excel(output_file, index=False)

print(f"✅ 중복 제거 완료! 오직 고유한 제목들만 담긴 '{output_file}' 파일이 생성되었습니다.")
```

중복데이터 삭제: 국립현대미술관 엑셀 파일을 불러와 중복 전시 항목들을 찾아낸 뒤, 가장 먼저 나온 데이터 1건만 남기고 나머지는 자동으로 삭제하는 코드다.

```

import pandas as pd
from datetime import datetime

# 1. period 열이 추가된 최신 엑셀 파일 불러오기
# (저장하신 파일 이름과 똑같은지 확인해 주세요!)
file_name = "culture_data_unique.xlsx"
df = pd.read_excel(file_name)

print(f"원본 데이터 수: {len(df)}개")

# 2. 'period' 컬럼에서 물결(~)을 기준으로 쪼갬 뒤, 뒤쪽의 '종료일'만 가져오기
# str.split('~') : ~를 기준으로 자름
# str[-1] : 자른 것 중 맨 마지막(오른쪽) 것을 선택
# str.strip() : 혹시 모를 양옆 띄어쓰기 여백을 깔끔하게 제거
df['end_date_str'] = df['period'].str.split('~').str[-1].str.strip()

# 3. 글자를 진짜 '날짜(Datetime)' 데이터로 변환하기
# errors='coerce'를 넣으면, 날짜 모양이 아닌 이상한 글자가 섞여 있어도 에러를 내지 않고 빈칸(NaT)으로 넘겨줍니다.
df['end_date'] = pd.to_datetime(df['end_date_str'], format='%Y-%m-%d', errors='coerce')

# 4. 오늘 날짜를 기준으로 과거 전시만 필터링하기
# 컴퓨터의 오늘 날짜를 가져옵니다 (시간은 빼고 날짜만).
today = pd.Timestamp('today').normalize()

# 종료일(end_date)이 오늘(today)보다 작다(<) = 이미 끝난 과거의 전시다!
df_past = df[df['end_date'] < today].copy()

# 보기 깔끔하게 임시로 만들었던 날짜 계산용 기둥 2개는 다시 지워줍니다.
df_past = df_past.drop(columns=['end_date_str', 'end_date'])

# 5. 결과 확인 및 저장
print(f"과거 전시(종료됨) 필터링 완료: 총 {len(df_past)}개 발견!")
print("-" * 30)

output_file = "culture_data_past_only.xlsx"
df_past.to_excel(output_file, index=False)

```

현재 진행중인 전시 제거: 중복이 제거된 엑셀 파일을 불러와 '202X-XX-XX~202X-XX-XX'와 같은 기간(period) 텍스트에서 물결표(~)를 기준으로 뒤쪽에 위치한 종료일 글자만 분리한다. 단순 문자열이었던 종료일을 컴퓨터가 날짜로 인식할 수 있는 날짜 데이터로 변환한 뒤, 현재 날짜와 대소 비교 연산을 수행한다. 이후 이미 끝난 과거 전시 데이터만 정확하게 선별하여 계산용 임시 컬럼을 정리한 후 파일로 최종 저장하는 코드다.

```

import pandas as pd

# 1. 파일 불러오기 (period 열이 있는 최신 파일 이름으로 맞춰주세요)
file_name = "culture_data_past_only.xlsx"
df = pd.read_excel(file_name)

print(f"원본 데이터 수: {len(df)}개")

# 2. 'period' 컬럼에서 물결(~)을 기준으로 쪼갬 뒤, 뒤쪽의 '종료일' 가져오기
df['end_date_str'] = df['period'].str.split('~').str[-1].str.strip()

# 3. 글자를 진짜 '날짜(Datetime)' 데이터로 변환하기
df['end_date'] = pd.to_datetime(df['end_date_str'], format='%Y-%m-%d', errors='coerce')

# 4. ★ 2024년 1월 1일 이후의 전시만 필터링하기
# 비교 기준이 될 '2024-01-01'을 진짜 날짜 데이터로 만듭니다.
target_date = pd.to_datetime('2024-01-01')

# 종료일(end_date)이 2024년 1월 1일과 같거나(>=) 그 이후인 것만 남깁니다!
df_2024 = df[df['end_date'] >= target_date].copy()

# 보기 깔끔하게 임시 기둥들은 다시 지워줍니다.
df_2024 = df_2024.drop(columns=['end_date_str', 'end_date'])

# 5. 결과 확인 및 저장
print(f"2024년 이후 전시 필터링 완료: 총 {len(df_2024)}개 발견!")
print("-" * 30)

output_file = "culture_data_2024_onwards.xlsx"
df_2024.to_excel(output_file, index=False)

```

24년도 이후 전시만 남기기: 변환된 데이터의 종료일이 2024년 1월 1일보다 같거나 큰 데이터만 선별함으로써, 너무 오래된 과거 데이터는 제외하고 '2024년 이후부터 현재까지' 진행되었던 데이터만 최종적으로 추출하여 파일로 저장하는 코드다.

<AI활용 프롬프트 정리>

Google Gemini Pro(5/4) 1차 프롬프트: (문화공공데이터포털 JAVA 활용 샘플 파일 제공) 해당 파일을 파이썬 형식으로 변환해 줘.

Google Gemini Pro(5/4) 2차 프롬프트: (전코드에서 데이터 출력 오류 발생: IOPub data rate exceeded) 해당 출력 오류를 해결하여 CSV파일을 제작하는 코드를 만들어줘.

Google Gemini Pro(5/4) 3차 프롬프트: (전코드 입력시 100개의 데이터로만 한정지어 파일 저장하는 상황 발생) API내 모든 자료를 활용할 수 있도록 반복문을 넣어서 코드를 제작해 줘.

Google Gemini Pro(5/4) 4차 프롬프트: (오류 해결) 원하는 Column(title, description, cntc_instt_nm, period)만 추출해서 csv 파일을 저장하는 코드를 제작해 줘.

Google Gemini Pro(5/4) 5차 프롬프트: 파일을 성공적으로 만들었어! 이제 저 파일 "cntc_instt_nm"이 '국립현대미술관'인 셀만 남겨두고 새로운 파일을 만들고 싶은데 코드를 알려줄 수 있어?

Google Gemini Pro(5/4) 6차 프롬프트: 'title'에서 겹치는 이름은 삭제하고 싶은데 그거 코드도 만들어줄래?

Google Gemini Pro(5/4) 7차 프롬프트: 이제 여기서 과거 전시만 뽑아내려고 해. 참고로 "2026-04-01~2026-12-06" 이런 식으로 열에 데이터가 입력되어 있어! 코드 제작해 줘.

Google Gemini Pro(5/4) 8차 프롬프트: 전시 종료 시점이 2024년 이후의 전시로 한정지어 줘. -> CSV파일 최종 완성

8-2. 전시 데이터 수집

2024년 이후 종료된 전시 데이터는 총 125개였으며, 이 중 동일한 전시가 서로 다른 명칭으로 중복 기재된 경우를 제외한 결과 최종적으로 95개의 전시 데이터로 정리되었다. 초기에는 오픈API에서 제공하는 <DESCRIPTION> 항목을 활용하여 전시별 키워드를 추출하고자 하였으나, 해당 항목의 내용이 전시마다 일관되지 않았고 누락값 및 오류값이 다수 확인되었다. 이에 따라 보다 정확한 전시 정보를 확보하기 위해 ‘국립현대미술관’ 홈페이지에서 제공하는 전시 정보를 기준으로 주요 내용을 수기로 정리하는 방식으로 변경하였다.

또한 데이터 분석 과정의 자동화를 위해 홈페이지 크롤링을 통한 데이터 수집도 고려하였으나, 사이트 접근 제한 및 보안상의 문제 등으로 인해 안정적인 데이터 수집이 어려웠다. 따라서 최종적으로는 국립현대미술관 홈페이지의 전시 설명 자료를 직접 확인하고, 분석에 필요한 정보를 수기로 수집한 뒤 정형데이터 형태로 정리하였다.

현재 진행 중인 전시 데이터 역시 앞서 설명한 데이터 수집 과정을 통해 확보하였다. 초기에

는 현재 진행 중인 전체 전시를 대상으로 데이터를 수집하고자 하였으나, 과거 전시 데이터를 2024년 이후 종료된 ‘국립현대미술관’ 전시로 한정하였기 때문에 추천 기준의 일관성을 유지하기 위해 현재 전시 또한 ‘국립현대미술관’ 전시로 범위를 제한하였다.

8-3. 키워드 뽑아내는 코드 과정

```
# [사전 준비] (코랩에서 최초 1회 실행)
!pip install konlpy

import pandas as pd
import re
import numpy as np
from konlpy.tag import Okt
from sklearn.feature_extraction.text import TfidfVectorizer
```

사전 준비: 데이터 분석에 필요한 한국어 형태소 분석기(KoNLPy, Okt)를 설치하고, 문장 속 단어들의 중요도를 계산하여 숫자로 바꿔주는 도구(TfidfVectorizer) 등 자연어 처리 알고리즘을 실행하기 위한 필수 라이브러리들을 불러오는 사전 준비 단계다.

```
# =====
# 0. 데이터 불러오기 및 통합
# =====
# 파일명 오류 방지를 위해 업로드하신 파일명을 그대로 사용합니다.
past_df = pd.read_excel("/content/26_1_데이터사이언스개론_팀플_과거전시.xlsx")
current_df = pd.read_excel("/content/26_1_데이터사이언스개론_팀플_현재전시.xlsx")

past_df['type'] = 'past'
current_df['type'] = 'current'

df = pd.concat([past_df, current_df], ignore_index=True)
df = df.dropna(subset=['description']).copy()
```

0단계: 저장한 과거, 현재 전시를 불러오고 이를 해당 데이터 분석에 활용할 수 있게 type 컬럼을 새로 만들어 라벨을 붙여준 후, 결측치를 제거하는 단계다.

```
# =====
# 1~3단계: 데이터 정제 및 형태소 분석
# =====
okt = Okt()
stopwords = [
    '전시', '작품', '작가', '미술관', '국립', '현대', '안내', '개최', '관람', '예술', '장소', '일시',
    '진행', '소장품', '소개', '제공', '이후', '중심', '과정', '의미', '스스로', '주요', '최근', '가지'
]

def clean_and_tokenize(text):
    text = re.sub('<[^>*>', ' ', str(text))
    text = re.sub('^가-힐Ws', ' ', text)

    words = okt.pos(text, stem=True)
    keywords = [word for word, pos in words if pos == 'Noun' and len(word) > 1 and word not in stopwords]
    return ' '.join(keywords)

df['tokenized_text'] = df['description'].apply(clean_and_tokenize)
```

1~3단계: 분석에 의미가 없는 단어들을 불용어 리스트로 지정한 후 문장에서 한 글자짜리 단어를 제외한 '두 글자 이상의 명사'만 골라낸 뒤, 앞서 설정한 불용어를 필터링하여 순수한 핵심 키워드들만 공백으로 연결된 문자열로 반환하는 단계이다. 이때 불용어를 선정한 기준은

“Gemini”에서 파일 확인 후 선정한 리스트이다.

```
# =====  
# 4단계: TF-IDF 기반 고유 핵심 태그(top_tags) 추출  
# =====  
tfidf = TfidfVectorizer()  
tfidf_matrix = tfidf.fit_transform(df['tokenized_text'])  
feature_names = np.array(tfidf.get_feature_names_out())  
  
def get_tfidf_top_tags(row_idx, top_n=5):  
    row_scores = tfidf_matrix[row_idx].toarray()[0]  
    top_indices = row_scores.argsort()[-top_n:][::-1]  
    top_words = feature_names[top_indices]  
    return ', '.join(top_words)  
  
df['top_tags'] = [get_tfidf_top_tags(i) for i in range(len(df))]
```

4단계: 이전 단계에서 정제한 명사 데이터를 바탕으로 TfidfVectorizer를 실행하여 문장 속 단어들을 숫자 행렬로 변환한다. 그 후 각 전시 데이터를 하나씩 확인하며 TF-IDF 점수가 가장 높은 상위 5개의 단어를 추출하는 함수를 한다. 추출된 고유 단어들을 쉼표(.)로 연결하여 새로운 컬럼에 저장함으로써, 각 전시의 성격을 가장 잘 나타내는 맞춤형 핵심 태그 셋을 자동으로 생성하는 단계다. 실제 문헌정보학에서 배우는 TF-IDF를 활용하여 코드를 제작했다는 점에서 의의가 있다고 생각한다.

8-4. 그룹화하는 코드 과정

```
# =====
# 5단계: 4대 장르 스코어링 및 상위 2개 제한 추출 (고도화 로직)
# =====
# 세부 장르들을 4가지 대분류(Macro-category)로 완벽히 통합했습니다.
taxonomy_dict_broad = {
    '매체 장르': ['회화', '한국화', '판화', '사진', '드로잉', '평면', '캔버스', '글씨', '서예', '한지',
                 '조각', '설치', '공예', '건축', '입체', '오브제', '공간', '물성', '양감', '조형물',
                 '미디어', '미디어아트', '영상', '사운드', '디지털', '인터랙티브', '애니메이션', '영화', '스크린'],

    '시대/성격 장르': ['유물', '보물', '역사', '민속', '고고학', '조선', '고대', '가야', '문화재', '전통', '불교',
                     '근대', '시대', '사건', '기록', '독립', '전쟁', '해방', '역사적',
                     '현대미술', '동시대', '실험', '개념', '추상', '새롭다', '아방가르드'],

    '기획 목적 장르': ['개인전', '회고전', '일생', '생애', '세계관', '거장', '조명', '선생', '유작',
                     '특별전', '기획전', '주제', '기획', '공동기획', '교류전',
                     '기증', '소장품', '컬렉션', '수장고', '수집', '기증품'],

    '경험 장르': ['어린이', '가족', '체험', '참여', '놀이', '워크숍', '교육', '체험형',
                 '아카이브', '기록물', '도면', '편지', '문헌', '자료', '도록', '인터뷰', '구술',
                 '물품', '감상', '분위기', '풍경', '빛', '색채', '공간', '묘화', '오감', '시각적']
}

def map_top_2_genres(keyword_string, top_n=2):
    words = keyword_string.split()
    genre_scores = {'매체 장르': 0, '시대/성격 장르': 0, '기획 목적 장르': 0, '경험 장르': 0}

    # 각 대분류별로 단어 출현 빈도를 점수화
    for word in words:
        for broad_genre, keywords in taxonomy_dict_broad.items():
            if word in keywords:
                genre_scores[broad_genre] += 1

    # 0점인 장르 필터링
    valid_scores = {k: v for k, v in genre_scores.items() if v > 0}

    if not valid_scores:
        return '기타/미분류'

    # 점수가 높은 순으로 정렬하여 상위 2개만 추출
    sorted_genres = sorted(valid_scores.items(), key=lambda item: item[1], reverse=True)
    top_genres = [genre for genre, score in sorted_genres[:top_n]]

    return ', '.join(top_genres)

df['genres'] = df['tokenized_text'].apply(map_top_2_genres)
```

5단계: 수많은 세부 키워드들을 4가지 대분류 기준(매체 장르, 시대/성격 장르, 기획 목적 장르, 경험 장르)으로 묶은 사전을 정의한 후, 각 전시의 키워드 텍스트를 읽어와 4대 장르에 속한 단어가 등장할 때마다 점수를 1점씩 부여하여 빈도를 계산한다. 단어가 하나도 매칭되지 않은 장르는 제외하고, 점수가 가장 높은 상위 2개의 대분류 장르만 추출하여 쉼표로 연결합니다. 만약 매칭되는 단어가 전혀 없을 경우 '기타/미분류'로 처리하며, 최종 결과를 genres 컬럼에 저장하여 전시의 성격을 거시적으로 분류한다.

```
# =====
# 6. 결과물 분리 및 엑셀 저장
# =====
final_df = df[['title', 'cntc_inst_nm', 'period', 'top_tags', 'genres', 'type']]

final_past = final_df[final_df['type'] == 'past'].drop(columns=['type'])
final_current = final_df[final_df['type'] == 'current'].drop(columns=['type'])

final_past.to_excel('최종_4대장르_과거전시.xlsx', index=False)
final_current.to_excel('최종_4대장르_현재전시.xlsx', index=False)
```

6단계: 데이터를 다시 과거와 현재로 나누고 그룹화 완료한 파일로 저장하는 단계다.

<AI활용 프롬프트 정리>

Google Gemini Pro(5/15) 1차 프롬프트: 전시에서 키워드를 뽑아내는 작업을 해야하는데 도움을 줘.

Google Gemini Pro(5/15) 2차 프롬프트: 상위어(장르/시소러스 사전)로 묶는 과정이 필요해. 이를 장르로 정리해줄 수 있어?

Google Gemini Pro(5/15) 3차 프롬프트: 시소러스 사전(장르)가 정교화가 안 되어서 정확도가 떨어지는 것 같아. 이를 해결해줘.

Google Gemini Pro(5/15) 4차 프롬프트: top_tag도 날카롭지 못하고 장르도 제한이 없어 변별력 부족 문제가 발생했어. 혹시 핵심 태그에 더 높은 가중치를 부여하는 TF-IDF 및 코사인 유사도 방식을 적용해서 코드를 만들어줄 수 있어?

Google Gemini Pro(5/15) 5차 프롬프트: top_tag는 날카워졌는데 장르에서 키워드 변별력이 떨어진다고 느껴져. 문제를 해결하기 위해서 지금까지의 피드백을 해주고 이를 반영해서 코드를 만들어줘.

Google Gemini Pro(5/15) 6차 프롬프트: 결론적으로 TF-IDF를 적용해서 top_tag를 변별력있게 추출하고, 장르를 4가지로 한정지어서 추출해줘. 이때, 매체 장르, 시대/성격장르, 기획 목적 장르, 경험 장르 4가지만으로 추출해주고, 최대 2개까지만 장르분야에 넣어줘.

9장. 프로그램 구현

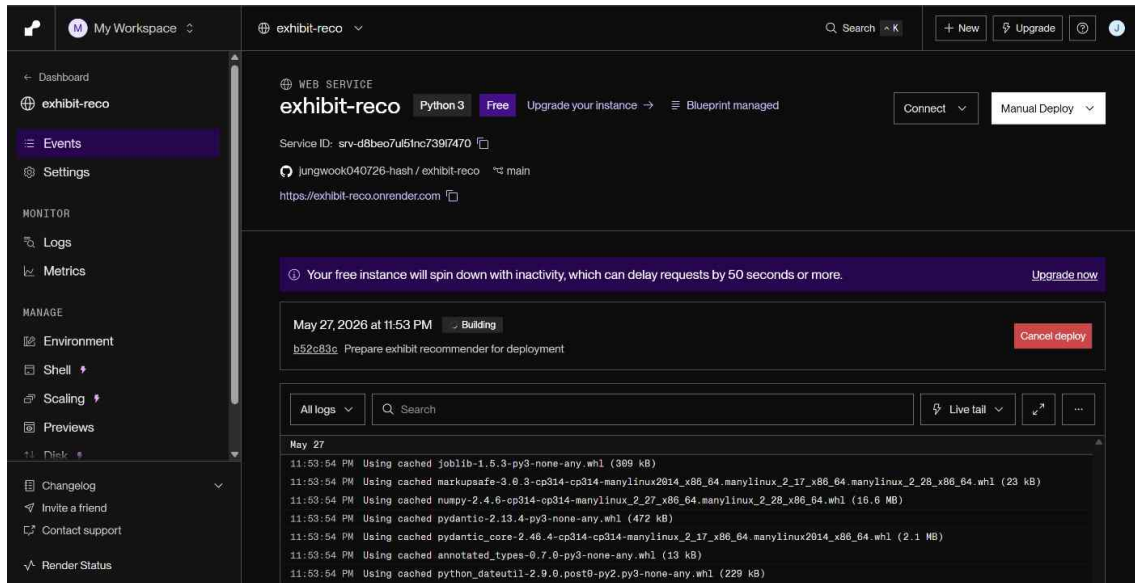
9-1. 1차 프로그램 - Cursor 사용(5/27 제작)

본 프로젝트에서는 수집하고 정리한 전시 데이터를 바탕으로, 과거 전시와 비슷한 현재 전시를 추천해주는 프로그램을 만들었다. 1차 프로그램 구현 과정에서는 Cursor를 활용하였다. Cursor는 코드를 작성하고 수정할 때 도움을 받을 수 있는 개발 도구이기 때문에, 추천 프로그램의 구조를 만들기 위한 과정에서 사용하였다.

이때, 프롬프트를 직접 입력하여 1차 프로그램을 구현하였으며, Cursor에서 제공해 준 코드를 터미널에 직접 입력하였다.

실제 프롬프트: “과거 전시를 토대로 현재 전시를 추천해 주는 프로그램 제작”, “현재 가지고 있는 데이터: 과거 전시, 현재 전시 키워드를 토대로 그룹화 완료”

프로그램 코드 제작은 완료되었으나, 인터넷을 통해 모두가 접속할 수 있게 배포하기 위해서는 Git, Github, Render 사이트를 활용하는 과정을 거쳐야 했다. 따라서 해당 사이트에 가입한 후 이를 Cursor에서 제시한 과정을 따라 제작한 코드와 사이트를 터미널을 통해 연결해 최종적으로 누구나 접속할 수 있게 배포 완료하였다.



(Render를 Github에 연결해 누구나 접속할 수 있게 배포하는 과정)

처음 프롬프트를 통해 제작된 프로그램은 한계점이 많았다. 따라서 수정사항을 담아, 새로운 프롬프트를 제작해 입력하였으며, 최종적으로 1차 프로그램 웹사이트를 제작했다.

실제 프롬프트: “사이트를 새롭게 만들어보고 싶어. 우리가 만든 사이트의 문제점은 <스코어가 0인 경우에는 추천 시스템이 제대로 작동하지 않음(순위 무효)> 그렇기 때문에 과거전시와 현재전시에 그룹화한 부분을 함께 사용해서 다시 추천 프로그램을 제작하고 싶어. 전시 파일을 보면 genres를 통해 크게 4가지를 나눴는데, 스코어도 함께 사용해서 프로그램을 다시 만들어줘”

<1차 프로그램 실제 화면 이미지>

과거 전시로 현재 전시 추천

past=95 (과거전시.xlsx) · current=14 (현재전시.xlsx)

과거 전시 선택
추천 개수 (1~100)

프로젝트 엑시테그 2023 (id=15)
10
추천 보기

추천 결과
최종점수 = TF-IDF + 장르 일치 + 그룹 일치 점수입니다.

- 로드 무비: 1945년 이후 한-일 미술**
id=0 · score=0.3000 · tfidf=0.0000 · genre=1.0 · group=0.0
genre: 시대 genre: 성격 장르 genre: 기획 목적 장르
교류 무비 로드 예술가 대형
- 방혜자 - 천지에 마음의 빛 뿌리며 간다**
id=2 · score=0.3000 · tfidf=0.0000 · genre=1.0 · group=0.0
genre: 시대 genre: 성격 장르 genre: 매체 장르
혜자 프랑스 일본 기법 원천
- MMCA 과천 상설전 «한국근현대미술 II»**
id=3 · score=0.3000 · tfidf=0.0000 · genre=1.0 · group=0.0
genre: 시대 genre: 성격 장르 genre: 기획 목적 장르
미술 한국 흐름 미술사 여성
- MMCA 과천 상설전 «한국근현대미술 I»**
id=4 · score=0.3000 · tfidf=0.0000 · genre=1.0 · group=0.0
genre: 시대 genre: 성격 장르 genre: 매체 장르
조선 시기 미술 실천 과천

(최초 제작된 1차 프로그램 프로토타입)

전시 추천

past=95 (과거전시.xlsx) · current=14 (현재전시.xlsx)

과거 전시 선택

다지털스토리 : 이야기가 필요해 (id=1)

▼

추천 개수 (1~10)

10

추천 보기

추천 결과

점수 모두 보기/숨기기

1. 개방 수장고 개편

id=13

장르: 매체 장르, 경험 장르

매체 장르, 경험 장르

공간 수장고 개편 조각

점수 숨기기

최종점수=0.5637 · TF-IDF=0.3972 · 장르일치=1.0

2. MMCA 다원예술 2026: 탐정의 시간

id=7

장르: 매체 장르, 경험 장르

매체 장르, 경험 장르

탐정 시간 관찰 미행 아이

점수 숨기기

최종점수=0.3000 · TF-IDF=0.0000 · 장르일치=1.0

3. 특별수장고: 국립현대미술관 드로잉·일본 현대 판화 소장품

id=11

장르: 매체 장르, 시대/성격 장르

매체 장르, 시대/성격 장르

판화 드로잉 특별 일본 장고

점수 숨기기

최종점수=0.2637 · TF-IDF=0.3972 · 장르일치=0.0

4. 로드 무비: 1945년 이후 한·일 미술

id=0

장르: 시대/성격 장르, 기획 목적 장르

시대/성격 장르, 기획 목적 장르

교류 무비 로드 예술가 대칭

5. 오~감각미술관

id=1

장르: 경험 장르, 매체 장르

경험 장르, 매체 장르

더욱 감각 만날 세계 걸음

6. 방혜자 - 천지에 마음의 빛 뿌리며 간다

id=2

장르: 시대/성격 장르, 매체 장르

시대/성격 장르, 매체 장르

혜자 프랑스 양본 기법 원전

(최종 제작된 1차 프로그램 프로토타입)

9-2. 2차 프로그램(최종) - Gemini pro 활용(5/28 제작)

1차 프로그램 프로토타입의 경우 우리가 생각했던 부분과 다른 방식으로 프로그램이 만들어졌다. 또한 “Cursor” 무료 버전의 한 달 사용량을 초과하였기에 새로운 생성형 AI를 활용해 코드를 제작하고 직접 터미널에 넣어 프로그램을 제작하기로 결정했다.

1차 프로그램 제작 당시 부족했던 부분을 고려해 프롬프트를 제작하였으며, 이를 Gemini Pro에 입력하였다.

실제 프롬프트: 전시 추천 프로그램을 “왓차피디아를 모방해 검색 시스템을 매칭하는 방식으로 만들어줘. 이때, <그룹과 태그를 계산해 매칭도를 계산, 과거 전시와 현재 전시 키워드를 토대로 추천, 과거 전시를 여러 개 선택해 현재 전시를 추천> 해당 부분을 고려해서 프로그램을 제작해 줘.”

이후 1차 프로그램 과정과 동일하게 github를 활용하였으며, 이후 Streamlit Cloud에 접속해 최종적으로 웹사이트를 제작하였다. 이때, 웹사이트 내 전시 추천 중 “취향 일치도” 기준은 아래의 기준 3가지를 고려하여 제작되었다.

1. 텍스트 특징 강화(가중치 부여)

단순히 데이터를 합치는 것이 아닌, ‘장르’보다 ‘태그’를 중요하게 반영하여 설계하였다. 코드 상 ‘genre’는 1번, ‘tag’는 3번 반복하여 구체적인 태그(세부 분위기)에 큰 가중치를 두었음을 알 수 있다. 일종의 가중치 부스팅 기법을 사용하였다.

2. TF-IDF를 이용한 단어 가치 산출

키워드를 뽑아낼 때와 동일하게 전시를 추천할 때도 TF-IDF 기법을 사용하였다. 만약 모

든 전시에서 흔하게 등장하는 단어라면 점수를 낮추고, 해당 전시의 개성을 잘 나타내는 희귀 단어라면 가중치를 크게 부여하였다.

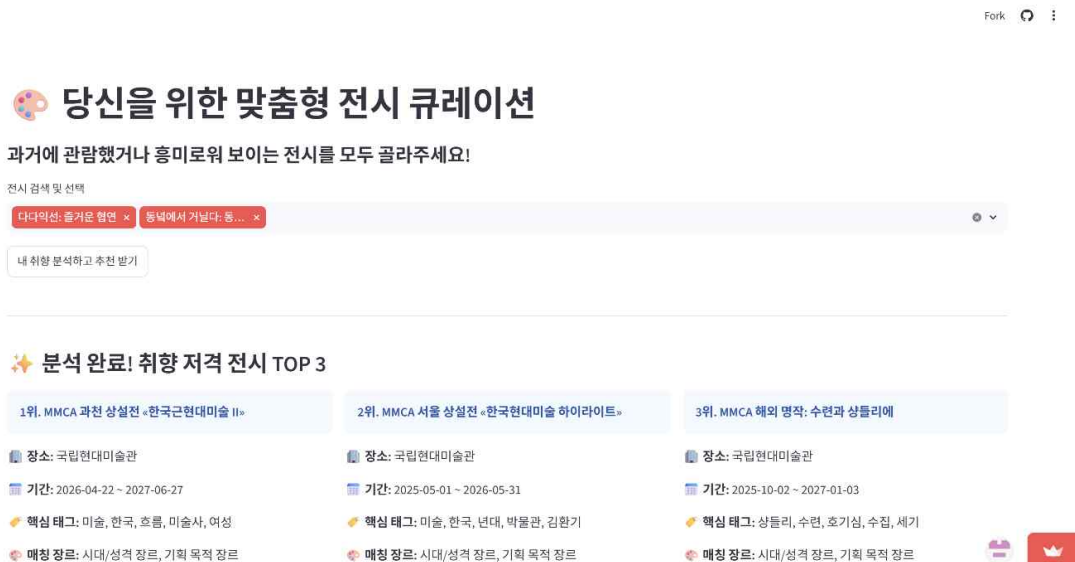
3. 코사인 유사도 활용

변환된 수치를 기반으로 계산을 수행하였다. 벡터화를 통해 사용자의 취향과 각 전시를 다차원 공간의 '벡터'로 만들었으며, 두 벡터 사이의 각도를 계산한 코사인 유사도를 활용하였다. 이때, 값이 1에 가까울수록(각도가 0도에 가까울수록) 두 전시가 같은 방향을 바라보고 있다고 판단해 높은 점수를 부여하였다.

<2차 프로그램 실제 화면 이미지>



(초기 화면)



(전시 선택 후 화면 1)

내 취향 분석하고 추천 받기

🌟 분석 완료! 취향 저격 전시 TOP 3

1위. MMCA 과천 상설전 «한국현대미술 II»

장소: 국립현대미술관

기간: 2026-04-22 ~ 2027-06-27

핵심 태그: 미술, 한국, 흐름, 미술사, 여성

매칭 장르: 시대/성격 장르, 기획 목적 장르

취향 일치도
12.5%

2위. MMCA 서울 상설전 «한국현대미술 하이라이트»

장소: 국립현대미술관

기간: 2025-05-01 ~ 2026-05-31

핵심 태그: 미술, 한국, 년대, 박물관, 김환기

매칭 장르: 시대/성격 장르, 기획 목적 장르

취향 일치도
12.5%

3위. MMCA 해외 명작: 수련과 상들리에

장소: 국립현대미술관

기간: 2025-10-02 ~ 2027-01-03

핵심 태그: 상들리, 수련, 호기심, 수집, 세기

매칭 장르: 시대/성격 장르, 기획 목적 장르

취향 일치도
2.9%



(전시 선택 후 화면 2)

9-3. 3차 프로그램(최종) - Gemini/groq 활용(6/2 제작)

2차 프로그램 버전의 경우 기존에 설계하였던 프로그램의 형태와 실행 양상은 전체적으로 양호하였으나, 활용하는 데이터의 종류가 수작업으로 만든 전시회 관련 키워드로 한정되어 다소 단조로운 데이터 분석 과정이라는 한계점이 나타났다. 때문에 프로그램 실행 과정 중간에 ai agent 서비스를 호출하여 ai가 사용자의 선호에 대한 데이터를 실시간으로 수집하고, 이를 추천 결과에 반영하는 아이디어를 내었고, 이를 실현하고자 하였다.

이후 1/2차 프로그램 과정과 동일하게 github를 활용하였으며, 이후 Streamlit Cloud에 접속해 최종적으로 웹사이트를 제작하였다.

키워드를 사용한 이용자 맞춤 추천 프로그램의 최종 코드 구조는 다음과 같다.

9-3-1. 코드 설계

```
import streamlit as st
import pandas as pd
import numpy as np
import requests
import json
# 💡 정밀한 취향 분석 알고리즘 복구
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

설명: 웹 대시보드(Streamlit)와 데이터 처리(Pandas), 외부 LLM 통신(Requests)을 위한 라이브러리이다. 수리 연산 모듈인 TfidfVectorizer와 cosine_similarity를 복구하였다. 단순 단

어 매칭이 아닌, 텍스트를 다차원 벡터 공간으로 변환해 각도를 계산하는 추천 엔진의 뼈대이다.

```
# =====
# 0. 무료 고속 AI (Groq API) Secrets 연동 호출
# =====
groq_api_key = st.secrets.get("keykey", None)
def call_llm_api(prompt):
    if not groq_api_key:
        return "ERROR: Streamlit Secrets에서 'keykey'를 불러올 수 없습니다."

    url = "https://api.groq.com/openai/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {groq_api_key}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "llama-3.1-8b-instant",
        "messages": [
            {
                "role": "system",
                "content": "너는 미술관 큐레이터야. 유저의 관심사를 파고드는 질문을 제
공해야 해. 인사말이나 서론, 결론은 모두 생략하고 오직 지정된 '질문:', '1.', '2.', '3.' 포맷
으로만 답변을 채워줘."
            },
            {"role": "user", "content": prompt}
        ],
        "temperature": 0.7
    }

    try:
        response = requests.post(url, headers=headers, json=payload, timeout=15)
        if response.status_code == 200:
            result = response.json()
            return result['choices'][0]['message']['content'].strip()
        else:
            return f"API_ERROR_CODE_{response.status_code}: {response.text}"
    except Exception as e:
        return f"EXCEPTION_ERROR: {str(e)}"
```

설명: groq를 호출하여 외부 거대언어모델(Llama-3.1-8b)과 통신하여 실시간으로 질문을 생

성하는 백엔드 파이프라인이다. Streamlit의 보안 기능인 st.secrets를 활용해 API Key(keykey)가 코드 외부에 안전하게 숨겨지도록 하는 인프라 보안 구조이다. system prompt 지정을 통해 AI가 쓸데없는 인사말을 생략하고 오직 질문과 3개의 객관식 보기 포맷만 깔끔하게 출력하도록 제어하였다.

```
# =====
# 1. 페이지 기본 설정 및 데이터 불러오기 (엑셀 & CSV 모두 지원하도록 방어)
# =====
st.set_page_config(page_title="MMCA 전시 큐레이션", page_icon="🎨", layout="wide")
@st.cache_data
def load_data():
    # 📁 엑셀과 CSV 중 있는 파일로 자동으로 안전하게 읽어오도록 개선
    try:
        past_df = pd.read_excel('최종_4대장르_과거전시.xlsx')
        current_df = pd.read_excel('최종_4대장르_현재전시.xlsx')
    except Exception:
        try:
            past_df = pd.read_csv('past.csv', encoding='utf-8')
            current_df = pd.read_csv('current.csv', encoding='utf-8')
        except Exception:
            past_df = pd.read_csv('past.csv', encoding='cp949')
            current_df = pd.read_csv('current.csv', encoding='cp949')

    # 결측치 처리
    for col in ['top_tags', 'genres']:
        if col in past_df.columns:
            past_df[col] = past_df[col].fillna('')
        if col in current_df.columns:
            current_df[col] = current_df[col].fillna('')

    return past_df, current_df
past_df, current_df = load_data()
```

설명: 전시회 메타데이터 파일(past, current)을 시스템으로 읽어오는 파트다. 핵심 포인트는 @st.cache_data 데코레이터를 사용하여 매번 화면이 바뀔 때마다 엑셀을 다시 읽지 않도록 메모리에 캐싱하여 속도를 극대화하였다. 파일 확장자가 엑셀(xlsx)이든 CSV이든, 인코딩이 utf-8이든 cp949이든 어떤 환경에서도 프로그램이 터지지 않고 안정적으로 구동되게끔 다중 예외 처리(Try-Except Loop)를 설계하였다.

```
# =====
# 2. 세션 스테이트(Session State) 초기화
```

```
# =====
if 'page' not in st.session_state:
    st.session_state.page = 1
if 'selected_titles' not in st.session_state:
    st.session_state.selected_titles = []
if 'current_q_idx' not in st.session_state:
    st.session_state.current_q_idx = 0
if 'questions_list' not in st.session_state:
    st.session_state.questions_list = []
if 'options_list' not in st.session_state:
    st.session_state.options_list = []
if 'user_answers_accumulated' not in st.session_state:
    st.session_state.user_answers_accumulated = []
st.title("☺ 당신을 위한 맞춤형 전시 큐레이션")
st.caption("선택하신 과거 전시별로 맞춤형 심층 질문을 드려 취향 파악 후, 최적의 현재 전
시를 추천합니다.")
st.markdown("---")
```

설명: 웹 브라우저가 새로고침되거나 사용자가 버튼을 누를 때, 기존에 선택한 데이터나 진행 상황이 날아가지 않고 유지되도록 임시 메모리에 저장하는 상태 관리 알고리즘이다. 사용자가 1페이지(전시 선택), 2페이지(질문답변) 3페이지(전시추천)으로 매끄럽게 이동하는 다단계 시퀀스(Multi-page Flow) 제어로 구성하였다.

```
# =====
# PAGE 1: 온보딩 (과거 전시 선택)
# =====
if st.session_state.page == 1:
    st.markdown("#### STEP 1. 과거에 관람했거나 흥미로워 보이는 전시를 골라주세요!
(최대 3개)")

    selected_past_titles = st.multiselect(
        "전시 검색 및 선택 (최대 3개까지 선택 가능)",
        options=past_df['title'].tolist(),
        default=st.session_state.selected_titles,
        placeholder="여기를 클릭하여 전시를 선택하세요"
    )

    if len(selected_past_titles) > 3:
        st.error("🚩 전시는 최대 3개까지만 고를 수 있습니다! 3개 이하로 조정해주세요.")
        selected_past_titles = selected_past_titles[:3]
    if st.button("내 취향 분석 질문 생성하기 ➡📝")
```

```

if not selected_past_titles:
    st.warning("최소 1개 이상의 전시를 선택해 주세요!")
elif not groq_api_key:
    st.error("🔗 Streamlit Secrets에 'keykey' 설정 상태를 확인해주세요.")
else:
    with st.spinner("선택하신 전시들 각각의 취향 분석 문항을 실시간으로 구성하  
는 중입니다..."):
        st.session_state.selected_titles = selected_past_titles

        generated_questions = []
        generated_options = []

        for title in selected_past_titles:
            single_df = past_df[past_df['title'] == title].iloc[0]
            user_tags = single_df['top_tags'] if 'top_tags' in past_df.columns
        else ''

            user_genres = single_df['genres'] if 'genres' in past_df.columns
        else ''

        prompt = f"""
유저가 전시회 제목과 직관적인 키워드만 보고 흥미를 느껴 선택한 [{title}] 전시 정보입니다.
- 관련 장르: {user_genres}
- 핵심 태그 키워드: {user_tags}
이 전시의 제목과 태그를 보고 이 유저가 '어떤 관심이나 취향'을 가지고 이 전시를 골랐을지
유추하여, 유저의 관심 분야를 한 단계 더 깊게 파고들 수 있는 '구체적인 질문 1개'와 '명확
한 객관식 선택지 3개'를 한국어로 만들어주세요.
[△중요 작성 규칙]
1. 전시회의 세부 내용이나 해설을 구구절절 묻지 마세요. 유저가 이 전시의 '장르나 태그'에
왜 매력을 느꼈는지, 그 '관심사' 자체에 중점을 두고 질문을 만드세요.
2. 제3자를 지칭하는 말은 절대 쓰지 마세요. 화면을 보고 있는 사람에게 직접 질문하듯 "~에
관심이 가시나요?", "~를 선호하시나요?"처럼 친근한 대화체로 작성하세요.
3. 선택지(1, 2, 3)는 유저가 본인의 취향에 맞춰 확실하게 고를 수 있는 형태로 작성하세요.
4. 잡설은 생략하고 반드시 아래 형태로만 출력하세요.
질문: [유저의 직관적 관심사를 파고드는 질문]
1. [선택지 1]
2. [선택지 2]
3. [선택지 3]
"""

        llm_output = call_llm_api(prompt)

```

```

        if llm_output and not any(x in llm_output for x in ["ERROR",
"API_ERROR", "EXCEPTION"]):
            lines = [l.strip() for l in llm_output.split('\n') if l.strip()]
            q_line = None
            options = []

            for line in lines:
                if "질문:" in line:
                    q_line = line.split("질문:")[1].strip()
                elif line.startswith("1.") or line.startswith("1번"):
                    options.append(line)
                elif line.startswith("2.") or line.startswith("2번"):
                    options.append(line)
                elif line.startswith("3.") or line.startswith("3번"):
                    options.append(line)

            if q_line and len(options) == 3:
                generated_questions.append(q_line)
                generated_options.append(options)
            else:
                generated_questions.append(f"[{title}] 전시의 어떤 분위기나
키워드에 가장 끌리셨나요?")
                generated_options.append(["1. 전통과 현대가 융합된 독창적
인 표현 기법", "2. 작품이 품고 있는 깊이 있는 시대적 메시지", "3. 전시가 전하는 고유한 감
성과 새로운 시각적 자극"])
            else:
                generated_questions.append(f"[{title}] 전시의 어떤 분위기나 키
워드에 가장 끌리셨나요?")
                generated_options.append(["1. 전통과 현대가 융합된 독창적인
표현 기법", "2. 작품이 품고 있는 깊이 있는 시대적 메시지", "3. 전시가 전하는 고유한 감성
과 새로운 시각적 자극"])

            st.session_state.questions_list = generated_questions
            st.session_state.options_list = generated_options
            st.session_state.current_q_idx = 0
            st.session_state.user_answers_accumulated = []
            st.session_state.page = 2
            st.rerun()

```

설명: 유저에게 과거 관람 전시를 입력받고, 그 전시의 메타데이터를 기반으로 LLM에게 맞춤형 질문 작성을 명령하는 핵심 기획 파트이다. 프롬프트 엔지니어링 내역은 단순히 질문을 달라고 한 것이 아니라 내부 코드를 통해 "과도하게 긴 해설 금지", "이용자 관심사 자체에만 집

중할 것", "제3자 어조 지양 및 친근한 대화체 사용" 등 엄격한 지침을 입력하였다. 규칙을 어겼을 때를 대비해 하드코딩된 '기본 질문 셋'을 예외 처리 밸브로 잠가두어 시스템 흐름이 멈추는 현상을 방지했다.

```
# =====
# PAGE 2: 개별 전시 질문 루프 단계
# =====
elif st.session_state.page == 2:
    idx = st.session_state.current_q_idx
    total_q = len(st.session_state.questions_list)
    current_title = st.session_state.selected_titles[idx]

    st.markdown(f"#### STEP 2. 당신의 취향 구체화 질문 ({idx + 1} / {total_q})")
    st.info(f"☺ 과거 전시 **[{current_title}]**에 대한 분석 문항입니다.")

    st.markdown(f"#### ? {st.session_state.questions_list[idx]}")

    user_choice = st.radio(
        "가장 마음에 드는 답변 방향을 골라주세요:",
        options=st.session_state.options_list[idx],
        key=f"q_radio_{idx}"
    )

    st.markdown("---")
    col1, col2 = st.columns(2)

    with col1:
        if st.button("◀ 처음으로 (전시 재선택)"):
            st.session_state.page = 1
            st.rerun()

    with col2:
        if idx == total_q - 1:
            if st.button("최종 전시 추천받기 🎯"):
                st.session_state.user_answers_accumulated.append(user_choice)

                with st.spinner("선택하신 모든 답변을 종합하여 알고리즘 유사도를 연산하고 있습니다..."):
                    user_selected_df =
                    past_df[past_df['title'].isin(st.session_state.selected_titles)]
```

```

        user_tags = " ".join(user_selected_df['top_tags'].tolist()) if
'top_tags' in past_df.columns else ''
        user_genres = " ".join(user_selected_df['genres'].tolist()) if
'genres' in past_df.columns else ''

        # 실시간 유저 선택지 단어를 추출해서 가중치 부여 (5번 반복)
        answer_keywords = ""
        ".join(st.session_state.user_answers_accumulated).replace('1.', '').replace('2.',
        '').replace('3.', '')

        answer_boost = (answer_keywords + " ") * 5

        # 최종 프로필 텍스트 및 현재 전시 피쳐 텍스트 생성
        user_profile_text = user_genres + " " + (user_tags + " ") * 3 + "
" + answer_boost

        current_df['features'] = current_df['genres'] + " " +
(current_df['top_tags'] + " ") * 3

        # 💡 원본 라이브러리(TF-IDF + Cosine) 연산 복구 및 적용
        tfidf = TfidfVectorizer()
        all_text = [user_profile_text] + current_df['features'].tolist()
        tfidf_matrix = tfidf.fit_transform(all_text)

        user_vector = tfidf_matrix[0:1]
        current_vectors = tfidf_matrix[1:]

        sim_scores = cosine_similarity(user_vector,
current_vectors).flatten()
        current_df['similarity'] = sim_scores

        st.session_state.top_recommendations =
current_df.sort_values(by='similarity', ascending=False).head(3)
        st.session_state.page = 3
        st.rerun()
    else:
        if st.button("다음 취향 질문 ➡️"):
            st.session_state.user_answers_accumulated.append(user_choice)
            st.session_state.current_q_idx += 1
            st.rerun()

```

설명: 화면에 나타나지 않는 사용자 정보 입력과 전시회 추천의 두 항목 사이의 핵심 알고리즘이다. 이때, 2차 프로그램 제작과 동일한 알고리즘을 사용하였다.(재설명)

1. 가중치 부스팅 (Weight Boosting)

유저의 실시간 답변은 텍스트 양이 적어 대형 전시 데이터에 파묻히기 쉽다. 이를 극복하고자 대분류 장르는 1배수, 핵심 태그는 3배수, 실시간 AI 답변 키워드는 5배수로 단어 빈도수를 조작하여 결합했다.

2. TF-IDF 행렬 변환

유저의 실시간 답변은 텍스트 양이 적어 대형 전시 데이터에 파묻히기 쉽다. 이를 극복하고자 대분류 장르는 1배수, 핵심 태그는 3배수, 실시간 AI 답변 키워드는 5배수로 단어 빈도수를 조작하여 결합했다.

3. 코사인 유사도 (Cosine Similarity)

변환된 유저 프로필 벡터와 현재 전시들의 피쳐 벡터 간의 '방향성(각도)'을 측정하여, 취향의 밀접도를 척도로 하여 1, 2, 3위를 정렬합니다.

```
# =====
# PAGE 3: 최종 추천 결과 화면
# =====
elif st.session_state.page == 3:
    st.balloons()
    st.markdown("### 💎 취향 종합 분석 완료! 당신을 위한 추천 전시 TOP 3")
    st.caption("유저님의 질문 답변 데이터가 고르게 반영된 개인 맞춤형 다이나믹 추천 결과입니다.")

    cols = st.columns(3)
    for idx, (i, row) in enumerate(st.session_state.top_recommendations.iterrows()):
        with cols[idx]:
            st.info(f"**{idx+1}위. {row['title']}**")

            # 유연한 컬럼명 매핑 처리
            inst_col = 'cntc_instt_nm' if 'cntc_instt_nm' in current_df.columns else
current_df.columns[1]
            period_col = 'period' if 'period' in current_df.columns else
current_df.columns[2]
            tags_col = 'top_tags' if 'top_tags' in current_df.columns else
current_df.columns[3]
            genres_col = 'genres' if 'genres' in current_df.columns else
current_df.columns[4]

            st.write(f"
```

```

st.write(f"🔍️📋**핵심 태그:** {row[tags_col]}")
st.caption(f"🎭 **매칭 장르:** {row[genres_col]}")

# 순수 다이나믹 유사도 점수 반영
st.metric(label="취향 일치도", value=f"{row['similarity'] * 100:.1f}%")

st.markdown("---")
if st.button("🔄 처음부터 다시 테스트하기"):
    st.session_state.page = 1
    st.session_state.selected_titles = []
    st.session_state.current_q_idx = 0
    st.session_state.questions_list = []
    st.session_state.options_list = []
    st.session_state.user_answers_accumulated = []
    st.rerun()

```

설명: 최종적으로 연산된 유사도 점수가 가장 높은 상위 3개의 전시를 화면에 배치하고, 가중치 부스팅과 코사인 유사도로 도출된 순수 수리적 일치율을 % 스코어로 시각화(st.metric)하여 유저에게 신뢰도 높은 개인화 결과를 제공하는 화면이다. 마지막으로 st.button() 함수를 통해 리셋 버튼을 누르면 세션 상태가 초기화되어 다시 처음부터 테스트할 수 있도록 하였다.

9-3-2. 효과

AI agent 기반 질문 답변 데이터 활용하는 기능을 추가함으로써 단순히 전시회 관련 비정형 데이터에서 추출한 데이터(키워드)를 통해서만 현재 전시회를 추천하는 것이 아닌, 추천 시스템 중간에 AI agent인 ‘Groq’의 무료 api를 불러와 이용자가 선택한 과거 전시의 키워드와 제목을 통해 질문과 답변 선택지를 추출하고, 이용자가 답변을 고르면 기존 현재 전시의 키워드 뿐만 아니라 이용자가 선택한 답변의 데이터 또한 추출 과정에 포함시켜 결과에 반영하도록 하였다. 이를 통해 보다 고차원적인 데이터 분석 결과를 도출할 수 있었다.

9-4. 프로그램 UI 제작 - imweb 활용(5/29 제작)

추천 프로그램의 기본 기능을 구현한 후에는 사용자가 프로그램을 보다 쉽게 이해하고 접근할 수 있도록 웹사이트 형태의 UI를 제작하였다. 기존에 구현한 추천 프로그램은 기능적으로는 작동하지만, 사용자가 바로 접하기에는 다소 개발 환경 중심의 결과물에 가까웠다. 따라서 최종 결과물을 단순한 코드 실행 화면이 아니라, 실제 서비스처럼 보이도록 구성하기 위해 아임웹을 활용하였다.

아임웹은 웹사이트를 비교적 쉽게 제작할 수 있는 도구로, 화면 구성과 디자인을 시각적으로 조정할 수 있다는 장점이 있다. 본 프로젝트에서는 아임웹을 활용하여 전시 추천 프로그램의 소개 페이지를 만들고, 이를 거쳐 최종 프로그램으로 이동할 수 있도록 설계하였다. 즉, 추천 기능 자체는 앞서 제작한 프로그램에서 수행되며, 아임웹으로 제작한 웹사이트는 사용자가 해당 프로그램에 접근할 수 있도록 연결해주는 역할을 한다.

이러한 UI 제작 과정은 프로젝트의 완성도를 높이는 데 중요한 역할을 하였다. 데이터 수집과 분석, 추천 로직 구현만으로도 프로그램의 기능은 완성될 수 있지만, 사용자가 실제로 사용하기 위해서는 프로그램의 목적과 사용 방법이 직관적으로 전달되어야 한다. 아임웹을 활용한 웹사이트 제작은 이 부분을 보완해주었으며, 분석 결과를 보다 사용자 친화적인 형태로 제시할 수 있게 하였다.

결과적으로 본 프로젝트는 단순히 전시 데이터를 수집하고 추천 로직을 만드는 것에서 끝나지 않고, 사용자가 실제 서비스처럼 접근할 수 있는 형태로 확장되었다. 아임웹으로 제작한 웹사이트는 추천 프로그램의 소개와 안내, 그리고 최종 프로그램으로의 연결 기능을 담당하였으며, 이를 통해 프로젝트 결과물을 보다 완성도 있는 전시 추천 서비스 형태로 구성할 수 있었다.

<AI활용 프롬프트 정리>

Google Gemini(6/2) 1차 프롬프트: 원래 취지가 “'과거 전시'를 통해서 '현재 전시'를 추천하자”야. 그래서 지금 과거전시 선택하면 해당 과거 전시에 할당된 키워드 제시하고 그걸 선택하면 현재전시 추천해 주는거야. 근데 중간에 키워드 제시가 아니라 키워드와 관련된 구체화된 질문으로 변경하면 좋을 것 같아. 만약에 '장난감'/'한국예술' 두 가지 각각 관련된 전시 선택했으면 '키덜트인가?'와 같은 질문하고 '한국예술 중에서 관심있는 분야 고르세요' 하고 선택지가 주는 식으로. 그래서 답변받은 다음에 그 답변 코사인 유사도 측정해서 관련있는 전시 추천해 주는거야. 과거 전시를 각각 고르는게 아니라 다 융합해서 하나로 취향을 도출한 다음에 그걸로 현재 전시 추천하는 시스템.

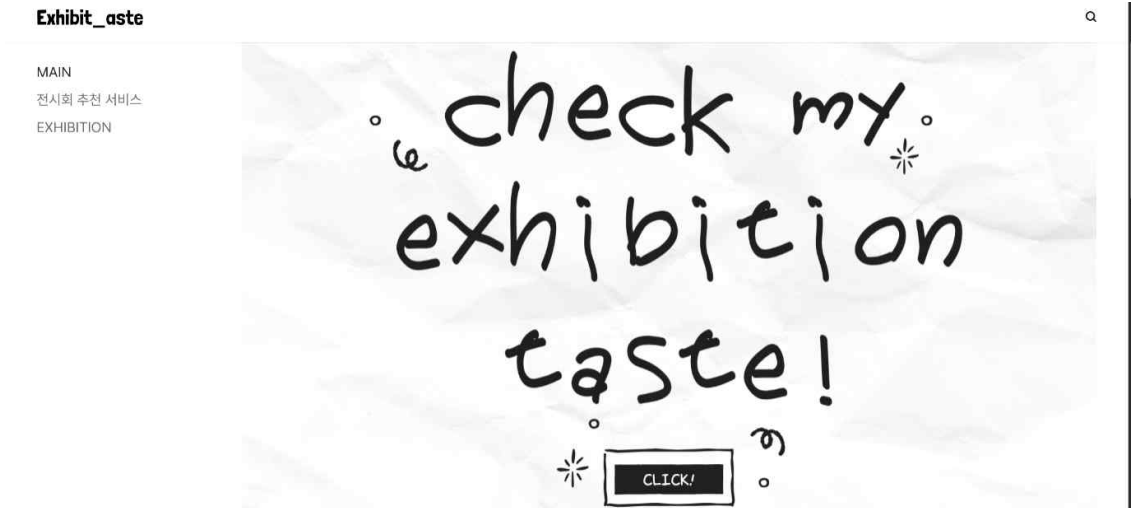
Google Gemini(6/2) 2차 프롬프트: 근데 이거 선택한 전시 키워드 종합한 다음에 질문하는거야 아니면 그냥 선택한 전시 중에 하나 골라서 질문하는거야? 만약에 후자이면 모든 전시 각각 질문 생성한 다음에 답변 종합해서 현재 전시 추천으로 해줘.

Google Gemini(6/2) 3차 프롬프트: 그냥 선택할 수 있는 전시 3개까지로 한정짓고, 각 전시회마다 질문 만들어서 전시회 개수만큼 질문하고 그 다음에 종합 -> 전시회 추천으로 수정해줘.

Google Gemini(6/2) 4차 프롬프트: 근데 이용자는 전시회 제목만 보고 선택하는거니까 내용 부분 말고 관심부분? 중점으로 질문 만들게 해줘.

Google Gemini(6/2) 5차 프롬프트: llm 불러오는 건 유지하면서 [코드 전문] 이게 기존 코드인데 이거랑 달라진 점 분석해줘.

<프로그램 UI 제작 이후 실제 화면 이미지>



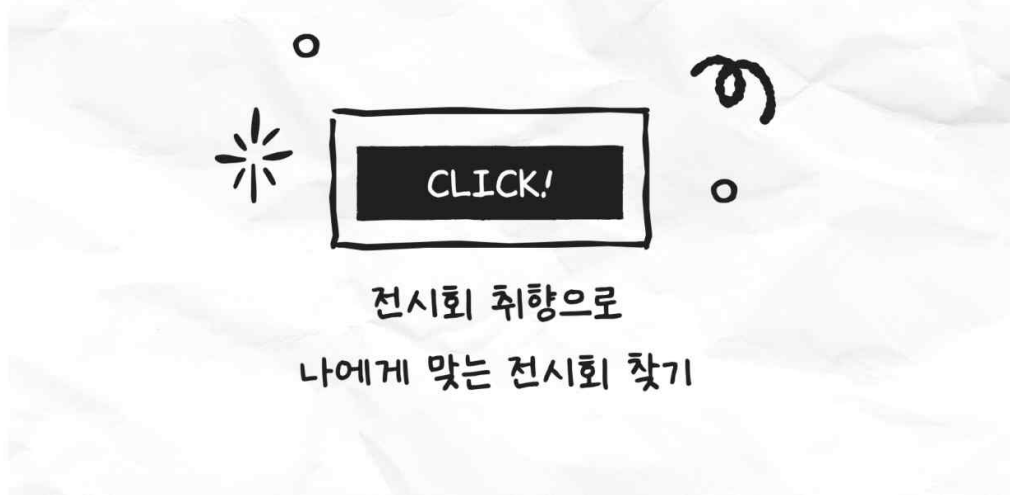
(MAIN 화면 1)



(MAIN 화면 2)



(MAIN 화면 3)



(전시회 추천 서비스 화면: 누를 시 2차 프로그램으로 넘어간다.)

MAIN
전시회 추천 서비스
EXHIBITION



(EXHIBITION 화면)

10장. 페르소나 테스트

10-1. 페르소나 테스트

프로그램 제작 이후에는 실제 사용 상황을 가정하여 페르소나 테스트를 진행하였다. 페르소나 테스트는 서로 다른 취향을 가진 가상의 사용자를 설정한 뒤, 이들이 기존에 선호하는 전시를 선택했을 때 프로그램이 어떤 전시를 추천하는지 확인하는 방식으로 이루어졌다.

이번 테스트에서는 총 세 명의 사용자를 설정하였다. 각각의 사용자는 관심 분야와 전시를 감상하는 방식이 다르게 나타나도록 구성하였다. 이를 통해 추천 프로그램이 모든 사용자에게 비슷한 전시를 추천하는 것이 아니라, 사용자의 취향과 선택 데이터에 따라 다른 결과를 보여 주는지 확인하고자 하였다.

10-2. 사용자 1: IT 기술에 관심이 많은 대학생

첫 번째 페르소나는 IT 기술에 관심이 많은 대학생으로 설정하였다. 이 사용자는 신기술과 감각적인 경험을 중요하게 생각하며, 전시를 볼 때도 단순히 작품을 감상하는 것보다 미디어 아트, VR·AR, 게임, 뉴미디어 등과 같이 시각적 자극이 강하고 현대적인 요소가 있는 전시를 선호하는 유형이다.

이 사용자가 선택한 과거 전시는 다음과 같다.

1. 《프로젝트 해시태그 2024》
* 태그: 해시태그, 프로젝트, 게임, 소망, 시물
2. 《MMCA 청주프로젝트 2023 - 안성석: 모두의 안녕을 위해》
* 태그: 미래, 도시, 기상, 게임, 픽션
3. 《순간이동》
* 태그: 영화, 인물, 순간이동, 중첩현실, 캐나다

위와 같은 선택을 바탕으로 추천된 현재 전시는 다음과 같다.

- 1위. 미술은행 20주년 특별전
- 2위. MMCA 과천 상설전: 한국근현대미술
- 3위. MMCA 서울 상설전: 한국 현대미술 하이라이트

추천 결과를 보면, 사용자 1에게는 미래, 게임, 기술, 중첩현실처럼 디지털·현대적 요소와 관련된 전시가 비교적 잘 반영되어 추천된 것을 확인할 수 있었다. 특히 1위로 추천된 미술은행 20주년 특별전은 여러 매체와 현대미술의 흐름을 함께 다루고 있어, 사용자가 선택한 전시의 키워드와 연결되는 부분이 있었다. 또한 한국근현대미술 및 현대미술 관련 상설전이 함께 추천된 것은 사용자의 취향이 단순한 기술 요소에만 머무르지 않고, 현대미술 전반의 시각적 실험성과도 연결되었기 때문이라고 볼 수 있다.

10-3. 사용자 2: 역사를 사랑하는 직장인

두 번째 페르소나는 역사를 사랑하는 직장인으로 설정하였다. 이 사용자는 한국 전통 회화, 역사적 자료, 수목화, 민속적 이미지 등에 관심이 많으며, 전시를 통해 시대적 배경이나 작품이 담고 있는 메시지를 이해하는 것을 중요하게 생각하는 유형이다.

이 사용자가 선택한 과거 전시는 다음과 같다.

1. 《수목별미》
* 태그: 동아시아, 수목, 채색화, 중국, 영국
2. 《미술관에 書: 한국 근대의 서예》
* 태그: 서예, 서예가, 글씨, 문자, 디자인
3. 《동네편에서 거닐다: 동산 박주환 컬렉션 특별전》
* 태그: 동산, 그림, 한국, 추환, 전통

위와 같은 선택을 바탕으로 추천된 현재 전시는 다음과 같다.

- 1위. MMCA 과천 상설전

2위. MMCA 서울 상설전

3위. MMCA 해외 명작: 수련과 샹들리에

사용자 2의 추천 결과에서는 ‘수묵’, ‘한국’, ‘역사’, ‘서예’, ‘전통’과 같은 키워드가 중요한 영향을 준 것으로 보인다. 1위로 추천된 MMCA 과천 상설전은 한국근현대미술의 흐름을 다루는 전시이기 때문에, 역사적 맥락을 중요하게 생각하는 사용자 2의 성향과 잘 맞는 결과라고 판단하였다. 2위로 추천된 MMCA 서울 상설전 역시 한국 현대미술의 흐름을 보여주는 전시라는 점에서 관련성이 있었다. 3위인 MMCA 해외 명작: 수련과 샹들리에에는 사용자가 선택한 전시와 직접적으로 동일한 주제를 다루지는 않지만, 미술사적 가치가 있는 작품을 감상할 수 있다는 점에서 추천 결과에 포함된 것으로 보인다.

10-4. 사용자 3: 감성이 풍부한 고등학생

세 번째 페르소나는 감성이 풍부한 고등학생으로 설정하였다. 이 사용자는 어렵고 심오한 해석보다는 전시를 보면서 편안함, 정서적 공감, 시각적 아름다움을 느끼는 것을 중요하게 생각하는 유형이다. 복잡한 설명보다 예쁜 공간, 자연적인 분위기, 따뜻한 색감, 직관적으로 감상할 수 있는 전시를 선호한다고 보았다.

이 사용자가 선택한 과거 전시는 다음과 같다.

1. 《정영선: 이 땅에 숨 쉬는 모든 것을 위하여》

* 태그: 조경, 정원, 정영선, 마당, 생태

2. 《MMCA 어린이미술관》

* 태그: 공공, 자연, 놀이, 시간, 어린이

3. 《MMCA 소장품 기획전 - 수채: 水彩》

* 태그: 수채, 장르, 단독, 투명, 습작

위와 같은 선택을 바탕으로 추천된 현재 전시는 다음과 같다.

1위. 오~ 감각미술관

2위. 미술은행 20주년 특별전

3위. 데이미언 허스트와 YBA

사용자 3의 추천 결과에서는 ‘공공’, ‘자연’, ‘수채’, ‘감성’과 같은 키워드가 주요하게 반영된 것으로 보인다. 1위로 추천된 오~ 감각미술관은 사용자가 선호하는 감각적이고 체험적인 전시 성향과 잘 맞는 결과였다. 특히 사용자가 선택한 전시들이 자연, 놀이, 색채, 정서적 경험과 관련이 있었기 때문에, 감각적으로 접근할 수 있는 전시가 가장 높은 순위로 추천된 것으로 해석할 수 있다. 2위인 미술은행 20주년 특별전은 다양한 현대미술 작품을 통해 시각적 경험을 제공한다는 점에서 관련성이 있었고, 3위인 데이미언 허스트와 YBA는 강한 시각적 요소와 대중적인 현대미술의 성격을 가지고 있어 사용자 3의 감성적 취향과 어느 정도 연결된다고 판단하였다.

10-5. 테스트 결과 정리

세 명의 페르소나 테스트를 통해 프로그램이 사용자별 선택 데이터에 따라 서로 다른 추천

결과를 보여주는 것을 확인할 수 있었다. IT 기술에 관심이 많은 사용자에게는 미래, 게임, 뉴미디어, 현대미술과 관련된 전시가 추천되었고, 역사와 전통에 관심이 많은 사용자에게는 한국근현대미술이나 미술사적 맥락이 있는 전시가 추천되었다. 또한 감성적이고 가벼운 전시 경험을 원하는 사용자에게는 감각적이고 체험적인 전시가 우선적으로 추천되었다.

이번 테스트를 통해 추천 프로그램이 단순히 전시 제목만 보고 결과를 내는 것이 아니라, 사용자가 선택한 과거 전시의 키워드와 장르를 바탕으로 현재 전시와의 유사성을 계산한다는 점을 확인할 수 있었다. 특히 사용자의 취향이 분명하게 드러나는 과거 전시를 선택할수록 추천 결과도 해당 성향에 맞게 나타나는 경향을 보였다.

다만 이번 페르소나 테스트는 실제 사용자를 대상으로 진행한 실험은 아니며, 가상의 사용자 유형을 설정하여 프로그램의 작동 가능성을 확인한 단계이다. 따라서 추천 결과가 실제 관람객의 만족도와 완전히 일치한다고 보기는 어렵다. 이후에는 실제 사용자를 대상으로 프로그램을 사용하게 한 뒤, 추천 결과가 얼마나 만족스러웠는지에 대한 평가를 함께 진행할 필요가 있다. 이를 통해 추천 알고리즘의 정확도와 실질적인 활용 가능성을 더 구체적으로 확인할 수 있을 것이다.

<AI활용 프롬프트 정리>

Google Gemini(6/2) 1차 프롬프트: 페르소나 테스트를 하고 싶은데, 제일 쉽게 할 수 있는 방법을 알려줘. 이용자 특성을 정하고 이에 맞게 선택지를 골라야 해. 그다음 나온 추천 전시회가 적절한지 확인해야 해.

Google Gemini(6/2) 2차 프롬프트: (과거전시.csv 제공) 이 파일을 기반으로 3명이 선택할 만한 전시 한 명 당 3개 이하로 정해주고 태그도 함께 작성해줘.

11장. 시행착오 및 버전관리

11-1. 프로젝트 타임라인

- 4/6: 주제 선정 회의 - 1차 주제는 “미술관, 박물관 전시 정보 및 큐레이션” or “공공데이터를 활용한 문제 해결” 2가지로 좁혀졌다.
- 4/10: 주제 선정 완료 - 최종적으로 “전시 추천 시스템”을 제작하는 것으로 결정하였다.
- 4/16: 보고서에 넣을 선행 연구 및 논문을 선정하였으며, 네이버 등 소셜미디어에서 제공되는 리뷰 데이터를 활용하여 분석 진행하기로 결정하였다.
- 5/3: 리뷰 데이터 수집 및 크롤링에 한계가 있어 문화공공데이터포털에서 제공하는 오픈 API를 활용하여 데이터 분석하기로 결정하였다.
- 5/6: API 활용하여 1차 데이터 분석 완료
- 5/14: 2차 데이터 분석 완료
- 5/21: 최종 키워드 및 그룹화 분석 코드 선정 완료
- 5/27: 프로그램 프로토타입 제작 완료

- 6/2 : 추천 시스템 수정 및 보완 (ai agent를 이용한 전시 선택에 대한 구체적인 질문 추가)

11-2. 시행착오 및 버전관리

1. v1.0 (5월 5일) : 초기 프로토타입 및 시소러스 구축 시도

구현 내용: 데이터 분석의 첫 시도로, 외부 엑셀 파일로 불용어 사전을 만들어 형태소 분석 (KoNLPy)을 진행하였다. 또한 전시 키워드를 묶기 위해 수작업으로 초기 시소러스(분류 사전) 매핑을 시도하였다.

한계 및 시행착오: 외부 엑셀 파일을 불러오는 과정에서 잦은 경로 오류가 발생하여 작업 효율이 떨어졌다. 또한 형태소 분석 시 동사, 부사 등에 대한 필터링이 정교하게 이루어지지 않아 핵심 키워드의 변별력을 확보하기 어려웠으며, 최종 출력 파일에 전시 제목과 태그만 남아 데이터 구조가 손실되는 문제가 발생하였다.

https://colab.research.google.com/drive/1kN9qay3NzNXgt1ZnJ2r6rtD3Q2X0_P1E?usp=sharing

2. v1.1 (5월 18일) : 데이터 파이프라인 개선 및 장르 체계 도입

구현 내용: 불안정했던 불용어 엑셀 파일을 삭제하고 코드 내부에 강력한 불용어 리스트를 직접 삽입(하드코딩)하여 연산 속도와 안정성을 높였다. 기존 시소러스를 발전시켜 '4대 장르(매체, 시대/성격, 기획 목적, 경험)' 체계를 확립하였고, 원본 데이터의 주요 정보(기간, 개최 기관 등)를 유지한 채 파일이 저장되도록 개선하였다.

한계 및 시행착오: 단어 추출 방식을 단순 빈도수(Counter)에 의존하다 보니 흔하게 쓰이는 단어가 핵심 태그로 잡히는 현상이 지속되었다. 또한 장르 분류 시 매칭되는 모든 장르가 텍스트로 산출되어, 한 전시에 3~4개의 장르가 무분별하게 부여되는 등 큐레이션의 기준이 모호해지는 문제가 발견되었다.

https://colab.research.google.com/drive/12rTWEdTZemoJUxt73kdkf0gx27VTG_Cy?usp=sharing

3. v2.0 (5월 21일) : 추천 알고리즘 고도화 (TF-IDF 도입)

구현 내용: 과거와 현재 전시를 별도로 분석하던 비효율적인 구조를 탈피하여, 하나의 데이터 프레임으로 통합 분석하는 파이프라인을 구축하였다. 단순 단어 빈도 계산을 넘어 정보검색론의 핵심 로직인 TF-IDF(단어 빈도-역문서 빈도) 기법을 전면 도입하여 각 전시만의 고유한 핵심 태그(top_tags)를 정교하게 추출해 냈다.

장르 스코어링 적용: 4대 장르 분류 시 단어 출현 빈도에 따라 점수(Score)를 부여하고, 점수가 가장 높은 상위 2개의 장르만 제한적으로 추출하도록 로직을 고도화하여 추천의 정확도를 대폭 끌어올렸다.

https://colab.research.google.com/drive/1L9PtC_NHNmpSG99H7T0t37wWxLTapBv5?usp=sharing

4. v2.1 (5월 25일) : 텍스트 마이닝 최종 최적화 (품사 필터링)

구현 내용: v2.0까지는 명사와 함께 형용사를 추출 대상으로 삼았으나, 전시 소개 글 특성상 형용사(예: 아름답다, 새롭다)가 데이터의 노이즈로 작용함을 파악하였다. 이를 해결하기 위해 형태소 분석 단계에서 철저하게 '명사(Noun)'만을 추출하도록 필터링 조건을 강화하였다.

최종 성과: 형용사가 배제됨에 따라 불용어 리스트 또한 군더더기 없이 경량화할 수 있었으며, 결과적으로 노이즈가 제거된 가장 순도 높은 명사 위주의 전시 키워드 데이터를 확보하여 최종 웹 서비스의 취향 매칭 정확도를 극대화할 수 있었다.

<https://colab.research.google.com/drive/1KWYlhDFtoib3nT1SpOceRJDTmD4XR-dF?usp=sharing>

5. v3.0 (6월 2일) : groq 활용 (ai agent)

구현 내용: v2.1까지는 단순 키워드를 활용한 추천 서비스로 다소 단조로운 내부 계산으로만 이루어져왔으나, open api로 ai agent 기능을 불러와 실시간으로 이용자의 선호 데이터를 받아와서 이를 키워드와 함께 계산 대상에 추가하는 로직을 통해서 보다 고도화된 추천 서비스를 실현시켰다.

https://colab.research.google.com/drive/1JMdaeu07NLRusSB_1DG5-8905QIIWJIV?hl=ko

12장. 한계점

1. 데이터 수집 및 전처리 과정에서의 높은 수작업 의존도

전시 정보를 웹사이트에서 직접 수집하고, 불용어를 수동으로 설정해야 했기 때문에 많은 시간이 소요되었다. 또한 수작업 과정에서 정보가 누락되거나 작업자의 판단이 개입될 가능성이 있어 데이터의 일관성과 재현성에 한계가 있었다.

2. 국립현대미술관 데이터로 한정된 분석 범위

전체 전시를 대상으로 하기에는 데이터의 양이 너무 많고 수집 범위가 넓어, 이번 프로그램에서는 국립현대미술관의 전시로 범위를 제한하였다. 따라서 본 프로그램은 전체 전시 정보를 포괄하는 완성형 서비스라기보다는, 특정 기관의 데이터를 활용하여 분석 및 추천 기능의 가능성을 확인한 프로토타입 형태라는 한계가 있다.

3. 최신 정보 반영 및 유지보수의 어려움

전시 일정, 장소, 소개 내용 등은 시간이 지나면서 변경될 수 있지만, 이를 자동으로 갱신하는 기능이 부족하다. 따라서 프로그램을 지속적으로 활용하기 위해서는 데이터를 주기적으로 확인하고 수정해야 하며, 이 과정에서도 추가적인 수작업이 필요하다.

13장. 느낀점

<유정욱>

이번 프로젝트는 주제 선정 단계부터 최종 결과물 제작까지 여러 차례의 방향 수정과 시행착오를 거치며 진행되었다. 처음에는 “미술관이나 박물관에 방문했을 때 소장품에 대한 정보가 충분하지 않다”는 문제의식에서 출발하였으나, 논의를 거듭하면서 최종적으로 전시 추천 프로

그램 제작이라는 주제로 구체화하게 되었다.

초기에는 네이버 전시 리뷰 데이터를 크롤링하여 프로그램에 활용하고자 하였지만, 사이트 보안 문제와 일부 전시의 리뷰 데이터 부족 등 현실적인 한계에 부딪히게 되었다. 이에 따라 데이터 수집 대상과 방법을 수정해야 했고, 프로젝트의 방향 역시 여러 차례 조정되었다. 특히 다른 조들이 주로 데이터 분석을 통해 문제의 현황을 파악하거나 결과를 도출하는 방식으로 진행한 것과 달리, 우리 조는 최종적으로 실제 프로그램을 제작하는 것을 목표로 하였기 때문에 과정이 더욱 막막하게 느껴지기도 하였다.

개인적으로도 데이터 분석과 프로그램 제작 경험이 많지 않은 상태에서 프로젝트를 진행해야 했기 때문에 어려움이 컸다. 오픈API를 불러와 필요한 데이터를 수집하고, 이를 코드로 정제하여 CSV 파일로 만드는 과정부터 처음에는 낯설게 느껴졌다. 또한 수집한 전시 데이터를 바탕으로 키워드를 추출하고 그룹화하는 코드를 설계하는 과정에서도 원하는 결과가 바로 나오지 않아 여러 번 수정과 실험을 반복해야 했다. 단순히 데이터를 모으는 것에서 끝나는 것이 아니라, 수집한 데이터를 분석에 적합한 형태로 정리하고, 이를 추천 프로그램에 활용할 수 있도록 구조화하는 과정이 중요하다는 점을 직접 경험할 수 있었다.

이후에는 Cursor를 활용하여 실제 전시 추천 프로그램을 제작하였다. 처음에는 코드의 흐름을 이해하고 수정하는 것조차 쉽지 않았지만, 오류를 하나씩 해결하고 기능을 구현해 나가면서 점차 프로그램의 구조와 데이터 처리 과정에 대한 이해가 깊어졌다. 특히 키워드 추출, 그룹화, 추천 로직 구현 과정에서 많은 시행착오가 있었지만, 그 과정을 통해 데이터 분석이 단순히 결과값을 도출하는 작업이 아니라 문제를 정의하고, 데이터를 정리하며, 적절한 방식으로 해석하고 활용하는 과정이라는 것을 깨닫게 되었다.

또한 팀장으로서 팀원들을 이끌어야 한다는 점 역시 부담으로 다가왔다. 그러나 팀원들과 지속적으로 소통하며 문제를 하나씩 해결해 나갔고, 바이브코딩을 활용하여 부족한 부분을 보완하면서 결과적으로 최종 프로그램을 완성할 수 있었다. 비록 예상보다 많은 시간이 소요되었지만, 직접 기획한 아이디어를 실제 결과물로 구현해냈다는 점에서 큰 성취감을 느낄 수 있었다.

이번 프로젝트를 통해 데이터 분석과 프로그램 제작에 대한 실질적인 경험을 쌓을 수 있었고, 처음에는 막연하게만 느껴졌던 데이터 수집, 정제, 분석, 구현의 흐름을 직접 체험하며 성장할 수 있었다. 또한 팀을 이끄는 과정에서 자연스럽게 리더십을 기를 수 있었고, 처음에는 데이터 분석과 프로그램 제작에 익숙하지 않았던 팀원들이 점차 구현 과정에 참여하고 성장하는 모습을 보며 협업의 중요성도 다시 한 번 느끼게 되었다. 무엇보다 이론적으로만 이해하는 것보다 직접 시도하고 부딪히는 과정이 훨씬 큰 배움으로 이어진다는 점을 깨닫게 된 의미 있는 경험이었다.

<김준규>

1. 데이터 분석 경험 요약

이번 프로젝트를 통해 로데이터(Raw Data) 수집부터 데이터 정제, 텍스트 마이닝, 추천 알고리즘 설계 그리고 웹 서비스 배포에 이르는 데이터 사이언스의 전 과정을 경험했습니다. 데이터 전처리 및 형태소 분석 단계에서는 KoNLPy를 활용해 텍스트를 형태소 단위로 분리하고, 전시 문맥을 해치는 무의미한 동사 및 수식어들을 정제하기 위해 불용어 리스트를 실시간으로 업데이트하며 노이즈를 제거했습니다. 추천 알고리즘 고도화 단계에서는 단순 빈도수 분석의 한계를 극복하기 위해 TF-IDF 알고리즘을 도입하여 범용 단어의 가중치를 낮추고 개별

전시 고유의 핵심 태그(top_tags)를 정교하게 도출했습니다. 마지막 분류 체계 및 스코어링 설계 단계에서는 추천의 변별력을 높이기 위해 주관적인 감성 언어 대신 4대 장르(매체, 시대/성격, 기획 목적, 경험)로 분류 체계를 객관화했습니다. 또한 하나의 전시 장르가 무분별하게 매칭되는 것을 막기 위해 '장르 스코어링 시스템'을 구축하여 점수가 높은 상위 최대 2개의 장르만 추출되도록 차원을 축소했습니다.

2. 프로젝트 느낀 점

첫째로는 “일단 부딪히면 된다”라는 실행력의 중요성을 알게 되었습니다. 프로젝트 초기에는 코딩 경험이 전무하여 데이터 분석에 대한 막연한 두려움과 거부감이 있었습니다. 하지만 구글 코랩 환경에서 기초적인 파이썬 코드를 실행해 보는 것을 시작으로 최종적으로는 GitHub에 코드를 업로드하고 Streamlit을 연동하여 실제 작동하는 웹 사이트까지 배포해 냈습니다. 이 과정을 통해 기술적 장벽은 부딪혀서 깨부술 수 있다는 큰 성취감과 자신감을 얻었습니다. 둘째로는 AI 활용의 핵심은 기획자의 '명확한 의도와 개입'이 필요하다는 점을 알게 되었습니다. AI는 훌륭한 보조 도구이지만, 개발자가 바라는 바가 명확하지 않다면 결코 프로젝트를 디벨롭할 수 없음을 느꼈습니다. 핵심 태그를 뾰족하게 만들기 위해 TF-IDF 기법을 AI에게 요구하거나, 무분별한 매칭 노이즈를 줄이기 위해 장르를 최대 2개로 제한하는 등 뚜렷한 의도와 지시가 존재할 때 비로소 AI가 유의미한 결과물을 도출해 낼 수 있음을 배웠습니다.

셋째로는 '버전 관리'의 필요성을 알게 되었습니다. 코드를 v1.0부터 지속적으로 업데이트하며 시행착오를 꼼꼼히 기록했습니다. 에러를 마주할 때마다 과정과 변화를 문서화해 두니 코드의 오류를 추적하기 쉬울 뿐 아니라 우리 시스템의 로직이 얼마나 올바른 방향으로 개선되고 있는지 객관적으로 파악할 수 있어 프로젝트의 방향성을 잃지 않는 데 큰 도움이 되었습니다. 넷째로 실무적 호환성을 위한 '영어 파일명'의 중요성을 알게 되었습니다. GitHub에 파일을 업로드하고 Streamlit 환경에 연동하는 과정에서 한글 파일명이나 경로 문제로 인한 인코딩 오류를 겪었습니다. 이를 통해 개발 환경에서는 파일명과 변수명을 영문으로 작성하는 것이 시스템 호환성과 안정성을 보장하는 가장 기본적이고 필수적인 규칙임을 직접 체감하며 배웠습니다.

마지막으로 팀 프로젝트의 책임감을 느꼈습니다. 데이터를 정제하고 낯선 알고리즘을 로직에 적용하는 과정은 결코 쉽지 않았고 많은 시간을 할애해야 했습니다. 만약 개인 프로젝트였다면 중간에 타협하거나 포기했을 수도 있었지만, 팀원들의 노력과 각자의 역할을 완수해야 한다는 팀 프로젝트의 '책임감'이 문제를 해결하는 강한 원동력이 되었습니다.

<배연진>

처음에는 장르와 태그의 가중치를 3대 7로 설정하여 하이브리드 추천 엔진을 설계했다. 이 과정에서 오픈 API 기반의 AI 에이전트 기능을 통합하다가 오류가 발생해, 기존의 TF-IDF와 코사인 유사도 연산 로직이 유실되었다. 당시 임시로 단순 단어 카운트 빈도(TF)만을 기준으로 추천을 대체했었는데, 사용자가 무엇을 선택하든 데이터 내에서 가장 흔하고 대중적인 특정 전시만 중복 추천되는 현상이 발생했다. 이후 다행히 시스템을 안정화하고 TF-IDF와 코사인 유사도 연산을 완벽히 복구해 냈고, 복구 전후의 추천 퀄리티 차이를 직접 눈으로 확인하면서 '개별 전시의 희소 가치를 반영하는 TF-IDF'와 '다차원 공간에서의 정밀한 각도를 계산하는 코사인 유사도'가 추천 시스템에서 왜 핵심적인 효용성을 가지는지 몸소 체감할 수 있었던 값진 경험이었다.

또한 단순히 AI를 호출하는 것에 그치지 않고, 코드 내부 프롬프트 작성을 통해 '이용자의

관심사에 집중할 것', '제3자 어조를 지양하고 친근한 대화체를 사용할 것' 등 세부적인 지시 사항을 설계했습니다. 다만, 한정된 시간 내에 적용하다 보니 AI가 생성한 질문의 가독성이나 출력 규격을 완벽하게 통제하는 최적화 단계에는 완벽히 도달하지 못한 아쉬움이 남는다. 만약 개발 초기부터 AI 에이전트 아키텍처를 상정하고 2달의 시간 동안 빌드업을 했었다면 훨씬 완성도 높은 UX를 구현했을 것이라는 깨달음을 얻었으며, 이는 향후 시대에 맞는 데이터 기반 서비스 시스템 설계에 대한 고찰로서 좋은 자산이 된 것 같다.